OnBuch+

SIGNED COURSE • PREMIUM

COMPUTER SCIENCE • UPPER SIXTH | UPPER SIXTH — COMPUTER SCIENCE

## CHAPTER 3

# APPLYING SEARCHING AND SORTING TECHNIQUES.

**SUGGESTED TIME: 2 PERIODS**

IN THIS CHAPTER

This chapter presents the fundamental searching algorithms (linear and binary search) and the classic sorting algorithms (bubble, selection, insertion), with their step-by-step traces, implementations and efficiency analysis. Students learn to compare algorithms and to select the appropriate technique for a given problem, a core skill for the GCE Advanced Level examination.

- By the end of this chapter I can explain what searching and sorting mean and why they matter in real applications.
- By the end of this chapter I can trace and implement a linear (sequential) search on a list of values.
- By the end of this chapter I can trace and implement a binary search and state the condition under which it works.
- By the end of this chapter I can compare the efficiency of linear and binary search using the number of comparisons.
- By the end of this chapter I can trace and implement bubble sort, selection sort and insertion sort on a small data set.
- By the end of this chapter I can count the number of comparisons and swaps performed by a sorting algorithm.


**DOWNLOAD ONBUCH**
Free • Google Play
AI tutor, past papers, results

**GO ONBUCH+**
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app


THIS SIGNED COURSE CONTAINS

**COURSE** complete, illustrated, step by step

**METHODS** and worked examples

**EXERCISES** exam style + solutions

**SUMMARY SHEET** the essentials to remember

CONTENTS

1. Searching Algorithms I: Linear Search
2. Searching Algorithms II: Binary Search
3. Sorting Algorithms I: Bubble Sort and Selection Sort
4. Sorting Algorithms II: Insertion Sort, Comparison and Choice of Algorithms
5. Integration activity
6. Methods and worked examples
7. Exercises
8. Solutions to the exercises
9. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can explain what searching and sorting mean and why they matter in real applications.
- By the end of this chapter I can trace and implement a linear (sequential) search on a list of values.
- By the end of this chapter I can trace and implement a binary search and state the condition under which it works.
- By the end of this chapter I can compare the efficiency of linear and binary search using the number of comparisons.
- By the end of this chapter I can trace and implement bubble sort, selection sort and insertion sort on a small data set.
- By the end of this chapter I can count the number of comparisons and swaps performed by a sorting algorithm.
- By the end of this chapter I can compare the time complexity of the studied algorithms using Big-O notation.
- By the end of this chapter I can choose the most appropriate searching or sorting algorithm for a given situation and justify my choice.
- By the end of this chapter I can write correct pseudocode and a program (e.g. in C or Python) for each studied algorithm.

PREREQUISITES

- Arrays (one-dimensional lists): declaration, indexing, traversal (Lower Sixth).
- Loops (for, while) and conditional structures (if…else).
- Writing and reading simple pseudocode and flowcharts.
- Basic notions of algorithm efficiency: counting elementary operations.
- Functions/procedures and parameter passing.

# 1 SEARCHING ALGORITHMS I: LINEAR SEARCH

Every morning at the Mokolo market in Yaoundé, a trader searching for one customer's name in a 500-page debt notebook flips page after page — unless she keeps the notebook sorted alphabetically, in which case she finds the name in seconds. The same drama plays out in every computer: a school database of 2 000 students, a phone contact list, the electoral register. How a program searches, and how it first organises its data, can mean the difference between an answer in a millisecond and an answer in an hour. In this chapter we discover the classic searching and sorting algorithms, measure their efficiency, and learn to choose the right one for the job.

## 1.1 What is Searching?

**DEFINITION** **Searching and the Search Key**

Searching is the process of determining whether a specific value, called the **search key** (or simply **key**), exists in a collection of data (an array, a list, a file, etc.) and, if it does, returning its position (index). If the key is not present, the algorithm reports failure (often by returning `-1` or a Boolean `false`).

In daily life we search constantly: a student looks for her name on an exam results sheet, a librarian finds a book by its call number, a phone contacts app locates a number when you type a name. In all these cases there is a collection (the results sheet, the catalogue, the contact list) and a target value (the name, the call number, the phone number). The efficiency of the search depends on how the collection is organised and on the algorithm used.

**DID YOU KNOW?** **Historical Note**

The concept of sequential search is as old as written records. Ancient Mesopotamian scribes used linear search on clay tablets to verify tax payments. In the 19th century, library catalogues used card drawers where a linear scan was the only option until Melvil Dewey's classification system (1876) introduced a sorted structure, enabling faster lookup — a physical analogue of moving from $O(n)$ to $O(\log n)$.

## 1.2 Linear (Sequential) Search: Principle

**DEFINITION** **Linear (Sequential) Search**

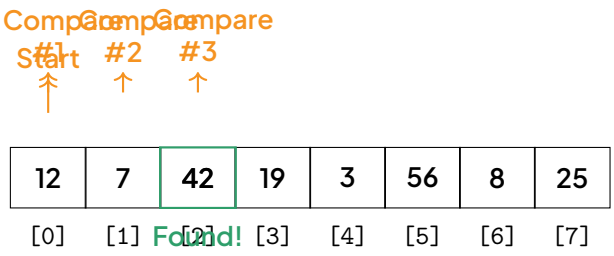
Linear search (also called sequential search) examines each element of the collection one by one, starting from the first, until either the key is found or the end of the collection is reached. It makes no assumption about the order of the data; it works on both sorted

and unsorted lists.

Imagine you have lost your student ID card in a classroom. You check each desk, one after another, until you find it or you have checked every desk. That is exactly linear search.

**EXEMPLE** **Linear Search on an Array of Integers**

Consider the array `A` of 8 integers below. We search for the key `42`.




Linear search for key 42: the algorithm checks indices 0, 1, then 2 where the key is found. Three comparisons were made.

Trace table:

| Step | Index examined | Element | Comparison result |
|---|---|---|---|
| 1 | 0 | 12 | 12 ≠ 42 → continue |
| 2 | 1 | 7 | 7 ≠ 42 → continue |
| 3 | 2 | 42 | 42 = 42 → key found at index 2 |

The search stops immediately when the key is found. If the key were `99` (not in the array), the algorithm would make 8 comparisons and then report “not found”.

## 1.3 Pseudocode and Implementation

**METHOD** **Linear Search Algorithm (returning index or –1)**

1. Input: array `A` of size `n`, search key `key`.
2. For `i` from `0` to `n-1` do:
3. If `A[i] == key` then return `i` (key found at index `i`)
4. End For

5. Return `-1` (key not found after examining all elements)

**EXEMPLE** **Pseudocode and Python Implementation**

Pseudocode (0-based indexing, standard for implementation):

```
FUNCTION linearSearch(A, n, key)
 FOR i = 0 TO n-1
 IF A[i] == key THEN
 RETURN i
 END IF
 END FOR
 RETURN -1
END FUNCTION
```

Python code:

```
def linear_search(arr, key):
 for i in range(len(arr)):
 if arr[i] == key:
 return i
 return -1
```

Note: In GCE pseudocode, 1-based indexing (`FOR i = 1 TO n`) is also accepted. If using 1-based indexing, adjust the return value accordingly (return `i` for position, `0` or `-1` for not found). Always state your indexing convention.

## 1.4 Variant: Linear Search on a Sorted List with Early Stop

If the list is sorted in ascending order, we can stop as soon as the current element exceeds the key, because all subsequent elements will be even larger.

**METHOD** **Linear Search on Sorted Array (Ascending)**

1. Input: sorted array `A` of size `n` (ascending), search key `key`.
2. For `i` from `0` to `n-1` do:

3. If `A[i] == key` then return `i`
4. Else if `A[i] > key` then return `-1` (key cannot appear later)
5. End For
6. Return `-1`

**EXEMPLE** **Early Stop on Sorted Array**

Array `B = [3, 7, 12, 19, 25, 42, 56, 78]` (sorted). Search for `20`.

| Step | Index | Element | Decision |
|---|---|---|---|
| 1 | 0 | 3 | 3 < 20 → continue |
| 2 | 1 | 7 | 7 < 20 → continue |
| 3 | 2 | 12 | 12 < 20 → continue |
| 4 | 3 | 19 | 19 < 20 → continue |
| 5 | 4 | 25 | 25 > 20 → stop, key not found |

Only 5 comparisons instead of 8. The early stop does not change the worst-case complexity (still $O(n)$) but improves the average case when the key is absent or located near the beginning.

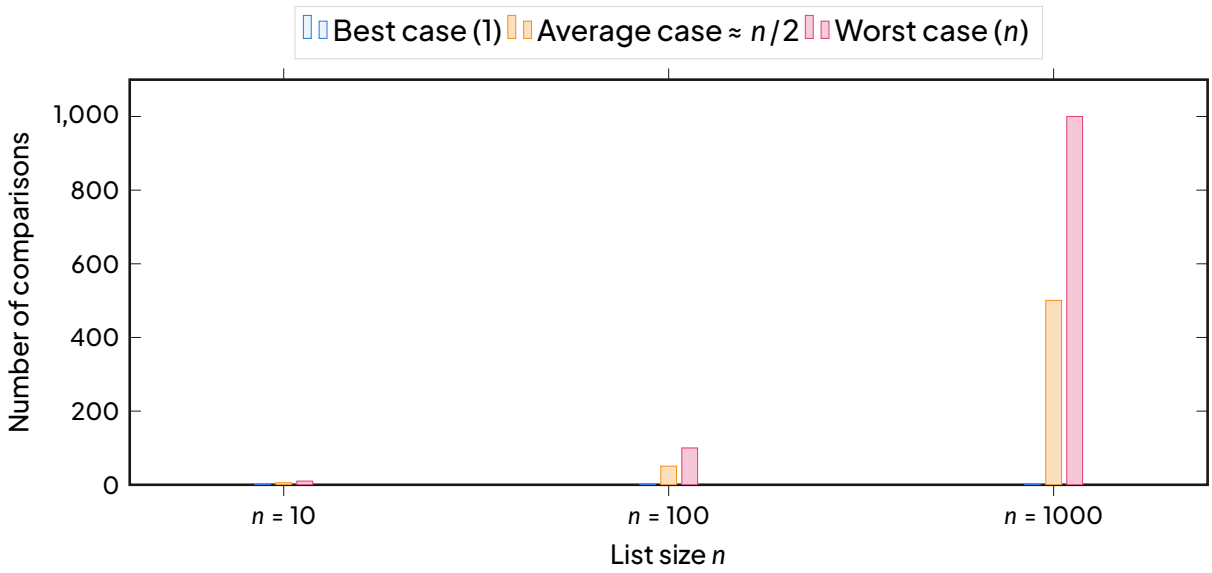
## 1.5 Efficiency Analysis

**PROPERTY** **Time Complexity of Linear Search**

For a list of `n` elements:

- Best case: 1 comparison (key is at the first position, index 0). $\Omega(1)$.
- Worst case: `n` comparisons (key is at the last position, index `n-1`, or not present). $O(n)$.
- Average case (successful search, uniform distribution): $\frac{n+1}{2} \approx \frac{n}{2}$ comparisons. $\Theta(n)$.
- Average case (unsuccessful search): `n` comparisons.

The time complexity is linear, denoted $O(n)$.




Comparison counts for linear search: best case is always 1, average grows as $n/2$, worst case grows as $n$.

## 1.6 Advantages and Limitations

**KEY POINTS** **Linear Search: When to Use It**

- Advantages:
  - Works on any list — sorted or unsorted.
  - Very simple to understand, implement, and debug.
  - No preprocessing (sorting) required.
  - Efficient for small data sets (e.g., $n < 50$) or when searches are infrequent.
- Limitations:
  - Slow for large data sets: doubling the list size doubles the search time.
  - Not suitable for real-time systems with large databases where $O(\log n)$ or $O(1)$ searches are needed.

## 1.7 Common Mistakes and Traps

**WARNING** **Frequent Errors in Linear Search**

1. Forgetting the "not found" case. If the loop ends without returning, you must return a sentinel value (`-1`, `None`, `false`) or raise an exception. Many students omit this and the function returns nothing (or garbage).
2. Off-by-one errors in the loop bounds. Using `i <= n` instead of `i < n` (0-based indexing) causes an out-of-bounds access. In 1-based pseudocode, the loop is `FOR i = 1 TO n`.
3. Modifying the loop variable inside the loop. Never write `i = i + 1` inside a `for` loop that already increments `i`.
4. Confusing the key with the index. The condition is `A[i] == key`, not `i == key`.
5. Using linear search on a large sorted list when binary search is possible. If the list is sorted and large, binary search ($O(\log n)$) is dramatically faster. Linear search on a sorted list is only justified for very small $n$ or when the list is stored on a medium that does not support random access (e.g., a tape drive).

## 1.8 Worked Examination-Style Exercise

**WORKED EXAMPLE 1** **Linear Search Trace and Complexity**

Statement: An array `SCORES` contains the marks of 12 students in a Computer Science test:

$$\texttt{SCORES} = [45, 62, 38, 71, 55, 62, 80, 49, 55, 67, 53, 60]$$

The pass mark is 50. A teacher wants to find the first student who scored exactly 62.

1. Trace the linear search algorithm for key = 62, showing the index examined and the comparison at each step until the key is found.
2. How many comparisons are made?
3. If the teacher instead searches for the key 90 (which is not in the array), how many comparisons will be made?
4. State the best-case, worst-case, and average-case time complexities of linear search for an array of size $n$.

**SOLUTION**

Detailed solution:

1. Trace table for key = 62:

| Step | Index $i$ | `SCORES[i]` | Comparison |
|---|---|---|---|
| 1 | 0 | 45 | $45 \neq 62 \rightarrow$ continue |
| 2 | 1 | 62 | $62 = 62 \rightarrow$ found at index 1 |

The algorithm stops at step 2 because the first occurrence of 62 is at index 1.

2. Number of comparisons: 2.
3. Search for key = 90 (absent): The algorithm examines all 12 elements (indices 0 to 11) and finds no match. 12 comparisons are made, then it returns -1.
4. Complexities for size $n$:
   - Best case: $\Omega(1)$ (1 comparison, key at index 0).
   - Worst case: $O(n)$ ($n$ comparisons, key at index $n - 1$ or absent).
   - Average case (successful, uniform): $\Theta(n)$ ($\approx n/2$ comparisons).

## 1.9 Summary

**KEY POINTS** **Key Points: Linear Search**

- Linear search scans the list sequentially from the beginning.
- It works on unsorted and sorted lists.
- Pseudocode: loop `i = 0 .. n-1`, if `A[i] == key` return `i`; after loop return `-1`.
- On a sorted list, add an early stop: if `A[i] > key` return `-1`.
- Time complexity: best $O(1)$, worst $O(n)$, average $O(n)$.
- Use for small lists or when the list is unsorted and you cannot sort it (or sorting would cost more than the search).
- Always handle the "not found" case and check loop bounds carefully.

**EXERCISE 1** **Practice Exercises**

1. Write a pseudocode function `linearSearchLast(A, n, key)` that returns the index of the last occurrence of `key` in `A`, or -1 if absent. Trace it on `A = [5, 2, 5, 8, 5]` with key = 5.

2. A sorted array `AGES` of length 20 contains the ages of students in ascending order. The teacher searches for age 17 using linear search with early stop. The array does not contain 17, but it contains 16 and 18. At which index will the search stop? Explain.
3. Compare the number of comparisons made by linear search (without early stop) and linear search with early stop when searching for a key that is larger than all elements in a sorted array of size $n$.
4. Programming task: Implement linear search in Python (or your preferred language) that returns a list of all indices where the key occurs. Test it on `[10, 20, 10, 30, 10]` with key 10.

**SOLUTION** **Exercise 1**

`linearSearchLast(A, n, key):`

- `lastPos = -1`
- `FOR i = 0 TO n-1`
- `  IF A[i] == key THEN lastPos = i`
- `END FOR`
- `RETURN lastPos`

Trace on `[5,2,5,8,5]`, key=5: i=0: A[0]=5 → lastPos=0; i=1: A[1]=2 → no change; i=2: A[2]=5 → lastPos=2; i=3: A[3]=8 → no change; i=4: A[4]=5 → lastPos=4. Returns 4.

**SOLUTION** **Exercise 2**

Since the array is sorted ascending and contains 16 and 18 but not 17, the early stop triggers when the first element > 17 is encountered. That element is 18. If 18 is at index `k`, the search stops at index `k` (after comparing `AGES[k]` with 17 and finding `AGES[k] > 17`). The exact index depends on the data, but it will be the position of the first element greater than 17.

**SOLUTION** **Exercise 3**

Key larger than all elements:

- Standard linear search: examines all $n$ elements → $n$ comparisons.
- Linear search with early stop: examines all $n$ elements because no element exceeds

the key until the end (actually, the condition `A[i] > key` is never true, so it also does $n$ comparisons). Both make $n$ comparisons.

**SOLUTION** **Exercise 4**

Python implementation:

```
def linear_search_all(arr, key):
    indices = []
    for i in range(len(arr)):
        if arr[i] == key:
            indices.append(i)
    return indices
```

Test: `linear_search_all([10,20,10,30,10], 10)` returns `[0, 2, 4]`.

# 2 SEARCHING ALGORITHMS II: BINARY SEARCH

In the previous section we studied linear search, a simple but brute-force technique that checks every element one by one. Its time complexity is $O(n)$, which becomes prohibitive when the data set grows to millions of records. Fortunately, if we invest the effort to keep our data sorted, we can exploit that order to eliminate half of the remaining candidates at each step. This is the idea behind binary search, also called dichotomous search, which reduces the worst-case number of comparisons from $n$ to approximately $\log_2 n$.

## 2.1 The Intuition: Divide and Conquer on Sorted Data

Imagine you are looking for the word “algorithm” in a printed dictionary. You do not start at page 1 and read every word. Instead, you open the book roughly in the middle. If the word on that page comes after “algorithm”, you know the target must be in the left half; otherwise it is in the right half. You then repeat the process on the chosen half, again jumping to its middle. Each step discards half of the remaining pages. This is exactly how binary search works on a sorted array.

**DEFINITION** **Binary Search (Dichotomous Search)**

Binary search is an efficient algorithm for finding a target value (the **key**) within a sorted array. It works by repeatedly dividing the search interval in half: compare the key with

the middle element; if they are equal, the search stops successfully; if the key is smaller, continue the search on the left sub-array; if it is larger, continue on the right sub-array. The process repeats until the key is found or the interval becomes empty.

DID YOU KNOW? **Divide and Conquer**

Binary search is the simplest example of a general algorithm design strategy called **divide and conquer**: break a problem into smaller sub-problems of the same type, solve them, and combine the results. The same strategy powers the fast sorting algorithms (merge sort, quick sort) that you will meet later in this chapter.

## 2.2 Precondition: The Data Must Be Sorted

This is the single most important condition for binary search. The algorithm **assumes** the array is sorted in ascending (or descending) order. If the data is not sorted, the logic "key < middle ⇒ go left" is meaningless, and the algorithm will either miss the target or return a wrong index.

WARNING **Binary Search Requires Sorted Data**

Never apply binary search on an unsorted collection. The result is undefined and usually incorrect. If your data is not sorted, you must either sort it first (which costs $O(n \log n)$ with a good sorting algorithm) or use linear search ($O(n)$). For a single search on unsorted data, linear search is faster overall because sorting takes longer than the search itself. Binary search becomes worthwhile when you perform many searches on the same sorted data, amortising the initial sorting cost.

## 2.3 The Binary Search Algorithm

We maintain two indices, `low` and `high`, that delimit the current search interval (inclusive). Initially `low = 0` and `high = n - 1`. At each step:

1. Compute `mid = (low + high) div 2` (integer division, rounding down).
2. Compare `key` with `A[mid]`.
3. If `key == A[mid]`: return `mid` (found).
4. If `key < A[mid]`: the key can only be in the left half ⇒ set `high = mid - 1`.
5. If `key > A[mid]`: the key can only be in the right half ⇒ set `low = mid + 1`.
6. Repeat while `low <= high`. If the loop ends, the key is not present; return `-1`.

**METHOD** **Computing `mid` and Updating the Interval**

1. Middle index: `mid = (low + high) div 2` (integer division, rounds down).
2. Key found: return `mid`.
3. Key smaller: discard the middle and the right half → `high = mid - 1`.
4. Key larger: discard the left half and the middle → `low = mid + 1`.
5. Loop condition: continue while `low <= high`.

**WARNING** **Infinite Loop Trap: Wrong Interval Update**

A classic mistake is to write `high = mid` or `low = mid` instead of `mid - 1` / `mid + 1`. If the key is not present and `mid` stops changing (for example when `low` and `high` become equal), the interval never shrinks and the loop runs forever. Always exclude the middle element, because you have just compared it and know it is not the key.

### Iterative Pseudocode

```
function binarySearch(A[0..n-1], key):
    low ← 0
    high ← n - 1
    while low <= high do
        mid ← (low + high) div 2
        if A[mid] == key then
            return mid
        else if key < A[mid] then
            high ← mid - 1
        else
            low ← mid + 1
        end if
    end while
    return -1   // not found
end function
```

**Recursive Version (Extension)** The same logic can be expressed recursively:

```
function binarySearchRec(A, key, low, high):
    if low > high then return -1
    mid ← (low + high) div 2
    if A[mid] == key then return mid
    else if key < A[mid] then
        return binarySearchRec(A, key, low, mid - 1)
    else
        return binarySearchRec(A, key, mid + 1, high)
end function
```

The recursive version uses $O(\log n)$ stack space, while the iterative version uses $O(1)$ auxiliary space. For GCE purposes, the iterative version is preferred unless recursion is explicitly requested.

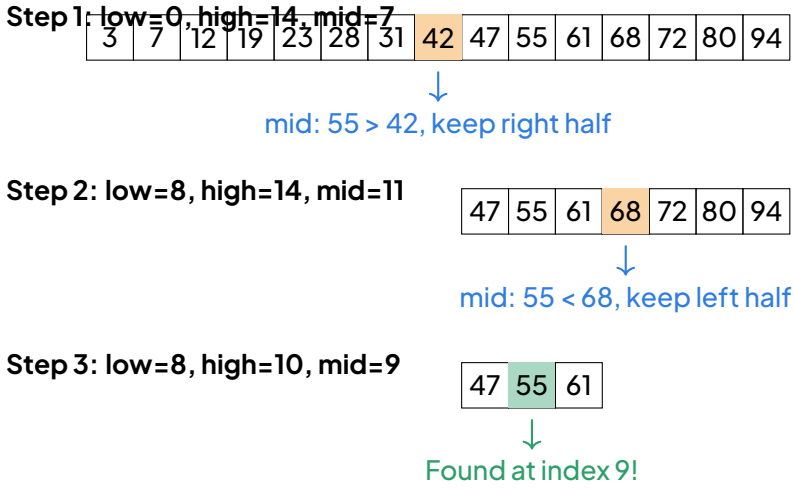
## 2.4 Tracing Binary Search: A Detailed Example

Let us trace the search for `key = 55` in the following sorted array of 15 elements (indices 0 to 14).

$$A = [3, 7, 12, 19, 23, 28, 31, 42, 47, \mathbf{55}, 61, 68, 72, 80, 94]$$

| Step | low | high | mid | A[mid] | Action |
|---|---|---|---|---|---|
| 1 | 0 | 14 | 7 | 42 | 55 > 42 → low = 8 |
| 2 | 8 | 14 | 11 | 68 | 55 < 68 → high = 10 |
| 3 | 8 | 10 | 9 | 55 | **Found at index 9** |

Only 3 comparisons were needed for an array of size 15. The figure below visualises how the search interval shrinks at each step.




The search interval halves at each step: 15 elements, then 7, then 3. The shaded cell is the middle element examined at that step.

In the worst case (key not present, or found only when the interval has shrunk to a single cell), the number of steps equals the number of times we can halve $n$ before reaching 1.

## 2.5 Efficiency Analysis: Logarithmic Time

At each iteration the search interval is halved. Starting with $n$ elements, after $k$ steps the interval size is at most $n / 2^k$. The process stops when the interval size drops below 1, i.e. when $n / 2^k < 1 \Rightarrow 2^k > n \Rightarrow k > \log_2 n$. Hence the maximum number of iterations is about $\lceil \log_2 n \rceil$. Each iteration does a constant amount of work (one comparison and a few assignments).

**PROPERTY** **Time Complexity of Binary Search**

- Worst case: about $\lceil \log_2 n \rceil$ comparisons $\in O(\log n)$.
- Best case: 1 comparison (the key is the middle element) $\in O(1)$.
- Average case: $O(\log n)$.
- Space complexity: $O(1)$ for the iterative version; $O(\log n)$ for the recursive version (call stack).

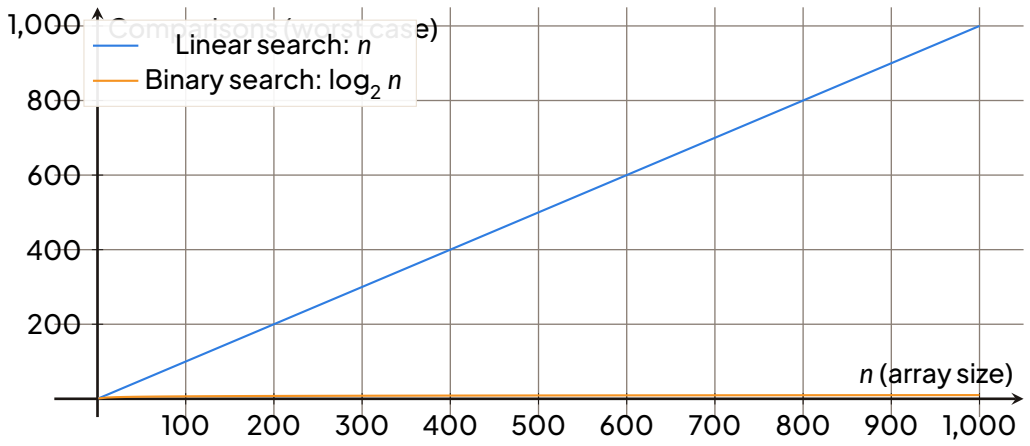
Examinable: you must be able to state the complexity and explain why it is logarithmic (the interval is halved at each step).

**KEY POINTS** **Numerical Comparison: Linear vs Binary**

For $n = 1\,000\,000$:

- Linear search worst case: 1 000 000 comparisons.
- Binary search worst case: $\lceil \log_2 1\,000\,000 \rceil = 20$ comparisons.

Binary search is roughly 50 000 times fewer comparisons in the worst case for one million elements. The gap widens dramatically as $n$ grows.




Growth of worst-case comparisons for linear ($n$) and binary ($\log_2 n$) search. The logarithmic curve is almost flat compared to the linear one.

## 2.6 When to Prefer Each Algorithm

| Criterion | Linear Search | Binary Search |
|---|---|---|
| Data order | Works on **unsorted** data. | **Requires sorted** data (ascending or descending). |
| Single search on unsorted data | **Better**: $O(n)$ versus $O(n \log n)$ to sort first. | Not applicable unless data is already sorted. |
| Multiple searches on same data | Poor: $O(m \times n)$ for $m$ searches. | **Excellent**: sort once in $O(n \log n)$, then $m \times O(\log n)$. |
| Small $n$ (e.g. $n < 50$) | Simpler, often fast enough. | Overhead of index calculations may not pay off. |
| Data structure | Works on arrays **and** linked lists (sequential access). | Requires **random access** (arrays); inefficient on linked lists. |
| Implementation complexity | Trivial. | Slightly more subtle (interval updates). |

## 2.7 Worked Example

**WORKED EXAMPLE 2** **Binary Search Trace and Complexity**

Statement: Consider the sorted array `A = [2, 5, 8, 11, 14, 17, 20, 23, 26, 29]` (indices 0 to 9).

1. Trace the execution of the iterative binary search algorithm for `key = 17`. Show the values of `low`, `high`, `mid`, `A[mid]` and the decision at each step.

2. How many comparisons are made in the worst case for an array of size 10? Justify using the formula.

3. Suppose we mistakenly write `high = mid` instead of `high = mid - 1` when `key < A[mid]`. Trace the search for `key = 1` (which is not in the array) and explain what happens.

SOLUTION

Detailed Solution:
1. Trace for key = 17

| Step | low | high | mid | A[mid] | Decision |
|---|---|---|---|---|---|
| 1 | 0 | 9 | 4 | 14 | 17 > 14 → low = 5 |
| 2 | 5 | 9 | 7 | 23 | 17 < 23 → high = 6 |
| 3 | 5 | 6 | 5 | 17 | Found at index 5 |

The algorithm terminates after 3 comparisons.
2. Worst-case comparisons for n = 10
$\lceil \log_2 10 \rceil = \lceil 3.32 \rceil = 4$. The worst case requires 4 comparisons (for example searching for a value that forces the algorithm down to an interval of size 1 before concluding). Indeed, we can halve 10 at most 4 times: 10 → 5 → 2 → 1 → 0.
3. Effect of the bug `high = mid`
Search for `key = 1` (smaller than all elements).

| Step | low | high | mid | A[mid] | Decision (buggy) |
|---|---|---|---|---|---|
| 1 | 0 | 9 | 4 | 14 | 1 < 14 → high = 4 |
| 2 | 0 | 4 | 2 | 8 | 1 < 8 → high = 2 |
| 3 | 0 | 2 | 1 | 5 | 1 < 5 → high = 1 |
| 4 | 0 | 1 | 0 | 2 | 1 < 2 → high = 0 |
| 5 | 0 | 0 | 0 | 2 | 1 < 2 → high = 0 |

From step 4 onward, `low = 0`, `high = 0`, `mid = 0` repeatedly. The condition `low <= high` remains true forever → infinite loop. The interval never becomes empty because the middle element is never excluded. This confirms the rule: always use `mid - 1` and `mid + 1`.

## 2.8 Summary of Key Points

KEY POINTS **Binary Search Checklist**

- Precondition: the array must be sorted (ascending or descending).

- Core idea: compare with the middle, discard half, repeat → divide and conquer.
- Iterative variables: `low, high, mid = (low + high) div 2`.
- Updates: `key < A[mid]` ⇒ `high = mid - 1`; `key > A[mid]` ⇒ `low = mid + 1`.
- Stop condition: `low > high` (not found) or `A[mid] == key` (found).
- Complexity: $O(\log n)$ time, $O(1)$ space (iterative).
- Use when: data is sorted and multiple searches are performed, or $n$ is large.
- Common bug: `high = mid` / `low = mid` causes an infinite loop.

**EXERCISE 2** **Practice: Binary Search**

1. Write the iterative binary search algorithm in pseudocode for an array sorted in descending order. Only the comparison direction changes.
2. An array of size 1000 is searched using binary search. What is the maximum number of comparisons? What is the maximum for an array of size 10000?
3. Critical thinking: You have an unsorted array of 10 000 integers and you need to perform exactly one search. Should you sort the array first (using an $O(n \log n)$ sort) and then use binary search, or just use linear search? Justify your answer with approximate operation counts.
4. Implement the iterative binary search in Python (or your preferred language) and test it with the array from the worked example, searching for keys 2, 29, 15, and 30.

**SOLUTION** **Exercise 1**

1. Descending order pseudocode: swap the two branches: when `key > A[mid]` set `high = mid - 1` (larger values are on the left), and when `key < A[mid]` set `low = mid + 1`.
2. Max comparisons: $n = 1000 \Rightarrow \lceil \log_2 1000 \rceil = 10$ (since $2^{10} = 1024 \geq 1000$). $n = 10000 \Rightarrow \lceil \log_2 10000 \rceil = 14$ (since $2^{14} = 16384 \geq 10000$).
3. One search on unsorted data: linear search costs at most 10 000 comparisons. Sorting first costs about $n \log_2 n \approx 10\,000 \times 14 = 140\,000$ operations, plus 14 for the binary search. Linear search is far better for a single search on unsorted data.
4. Implementation: standard iterative code; the test cases should return indices 0, 9, -1, and -1 respectively.

# 3 SORTING ALGORITHMS I: BUBBLE SORT AND SELECTION SORT

In the previous section we discovered the power of **binary search**: it can locate a value among a million records in about twenty comparisons. But binary search has one strict condition — the data must already be sorted. A natural question therefore arises: how do we get the data into sorted order in the first place? That is exactly the problem this section solves. We study the two simplest sorting algorithms, **bubble sort** and **selection sort**, which are the foundation for everything else you will meet in this chapter.

## 3.1 What is Sorting?

Imagine the GCE Board has just received the results of 5 000 candidates in a school. The marks are stored in the order the scripts were marked — completely mixed up. To publish a merit list, to find the pass mark, or simply to answer the question "who came first?", the marks must be arranged in order. This operation is called **sorting**.

**DEFINITION** **Sorting, ascending and descending order**

**Sorting** a collection of data means rearranging its elements so that they follow a well-defined order:

- **ascending order**: each element is less than or equal to the next one, e.g. 2, 5, 7, 9, 12;
- **descending order**: each element is greater than or equal to the next one, e.g. 12, 9, 7, 5, 2.

In this course, unless stated otherwise, we sort in ascending order.

**DEFINITION** **Swap (exchange)**

A **swap** is the operation that exchanges the contents of two variables (or two positions of an array). Swapping $A[i]$ and $A[j]$ requires a temporary variable:

```
temp ← A[i]

A[i] ← A[j]

A[j] ← temp
```

**WARNING** **The classic swap trap**

Writing `A[i] ← A[j]` then `A[j] ← A[i]` does not swap the values: after the first line, both positions contain the old value of `A[j]`, and the old value of `A[i]` is lost forever. The tem-

porary variable is compulsory. This is a favourite GCE trap question.

Why does sorted data matter so much?

- **Fast searching**: binary search (previous section) only works on sorted data.
- **Reports and rankings**: merit lists, league tables, price lists, telephone directories.
- **Data presentation**: humans read ordered information far more easily.

Throughout this section we use four key words: a **comparison** tests the order of two elements (e.g. `A[j] > A[j+1]`); a **swap** exchanges them; a **pass** is one complete sweep through (part of) the array; and as the algorithm progresses, the array splits into a **sorted region** (already in final order) and an **unsorted region** (still to be processed).

## 3.2 Bubble Sort

**Intuition.** Fill a glass with Fanta and watch the bubbles: the lightest bubbles rise to the top, one after another. Bubble sort works the same way. We sweep through the array comparing adjacent pairs; whenever a pair is out of order, we swap it. At the end of the first pass, the largest element has "bubbled up" to the last position. After the second pass, the second largest is in the second-to-last position, and so on.

DEFINITION **Bubble sort**

**Bubble sort** repeatedly scans the array, comparing each pair of adjacent elements and swapping them if they are in the wrong order. After pass number $k$, the $k$ largest elements occupy their final positions at the right end of the array (the sorted region), so each new pass can stop one position earlier.

EXEMPLE **Full trace of bubble sort**

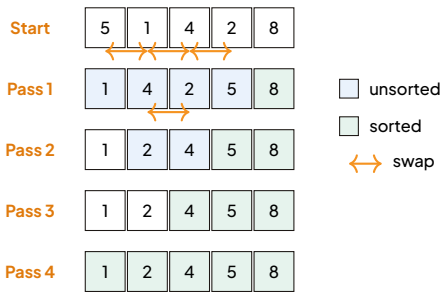
Sort the array $A = [5, 1, 4, 2, 8]$ in ascending order. We write C for a comparison and S for a swap.
Pass 1 (compare positions 1–2, 2–3, 3–4, 4–5):

| Comparison | Result | Swap? | Array after |
|---|---|---|---|
| 5 vs 1 | 5 > 1 | Yes | [1, 5, 4, 2, 8] |
| 5 vs 4 | 5 > 4 | Yes | [1, 4, 5, 2, 8] |
| 5 vs 2 | 5 > 2 | Yes | [1, 4, 2, 5, 8] |
| 5 vs 8 | 5 < 8 | No | [1, 4, 2, 5, **8**] |

After pass 1: 4 comparisons, 3 swaps; the largest value 8 is fixed at the end.
Pass 2 (positions 1–2, 2–3, 3–4 only):

| Comparison | Result | Swap? | Array after |
|---|---|---|---|
| 1 vs 4 | 1 < 4 | No | [1, 4, 2, 5, 8] |
| 4 vs 2 | 4 > 2 | Yes | [1, 2, 4, 5, 8] |
| 4 vs 5 | 4 < 5 | No | [1, 2, 4, **5**, 8] |

After pass 2: 3 comparisons, 1 swap; 5 is fixed.
Pass 3 (positions 1–2, 2–3): 1 < 2 (no swap), 2 < 4 (no swap). 2 comparisons, 0 swaps; 4 is fixed.
Pass 4: 1 < 2 (no swap). 1 comparison; 2 is fixed. The array is sorted.
Totals: 4 + 3 + 2 + 1 = 10 comparisons, 4 swaps. Note that $10 = \frac{5 \times 4}{2} = \frac{n(n-1)}{2}$.




Bubble sort on [5, 1, 4, 2, 8]: at each pass the largest unsorted value bubbles to its final position (green). Orange arrows show the swaps performed during the pass.

**METHOD** **Tracing a sorting algorithm (exam technique)**

1. Draw a table with one row per pass.
2. In each row, write the full array as it stands at the end of that pass.
3. Shade or underline the sorted region (it grows by one cell per pass).
4. Count the comparisons and swaps of the pass in two extra columns.
5. Check at the end: the sorted region must never change again.

**Pseudocode of bubble sort (basic version).** The array $A$ has $n$ elements, indexed from 1 to $n$.

```
FOR i ← 1 TO n - 1 DO
  FOR j ← 1 TO n - i DO
    IF A[j] > A[j+1] THEN
```

```
      temp ← A[j] ; A[j] ← A[j+1] ; A[j+1] ← temp
    END IF
  END FOR
END FOR
```

Notice the inner bound `n - i`: after pass $i$, the last $i$ elements are already in place, so we must not touch them again.

**Optimised version (early stop).** In our trace, pass 3 made no swap at all. That means the array was already sorted — yet the basic version still ran pass 4! We add a Boolean flag: if a pass performs no swap, we stop immediately.

```
i ← 1
REPEAT
  swapped ← FALSE
  FOR j ← 1 TO n - i DO
    IF A[j] > A[j+1] THEN
      temp ← A[j] ; A[j] ← A[j+1] ; A[j+1] ← temp
      swapped ← TRUE
    END IF
  END FOR
  i ← i + 1
UNTIL swapped = FALSE OR i = n
```

**WARNING** **Loop bounds: the most common mistake**

The inner loop of bubble sort must shrink after each pass (`j` goes from 1 to $n - i$, not to $n$). If you always loop to $n$, two things go wrong: you waste time re-comparing elements that are already in place, and worse, `A[j+1]` eventually reads beyond the end of the array — an out-of-range error. In the GCE, always show the shrinking bound.

## 3.3 Selection Sort

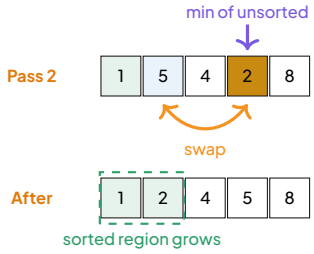
**Intuition.** A teacher arranging students by height does it differently: she scans the whole line, picks out the shortest student, and places him at the front. Then she scans the remaining students, picks the shortest of those, and places him second. And so on. That is exactly selection sort.

**DEFINITION** **Selection sort**

**Selection sort** divides the array into a sorted region (initially empty, on the left) and an unsorted region. At each pass, it finds the smallest element of the unsorted region and swaps it with the first element of the unsorted region. The sorted region then grows by one element. After $n - 1$ passes, the array is sorted.

**EXEMPLE** **Full trace of selection sort**

Same array $A = [5, 1, 4, 2, 8]$. The sorted region is shown in bold.
Pass 1: minimum of $[5, 1, 4, 2, 8]$ is 1 (position 2). Swap with position 1: $[\mathbf{1}, 5, 4, 2, 8]$. (4 comparisons, 1 swap)
Pass 2: minimum of $[5, 4, 2, 8]$ is 2 (position 4). Swap with position 2: $[\mathbf{1, 2}, 4, 5, 8]$. (3 comparisons, 1 swap)
Pass 3: minimum of $[4, 5, 8]$ is 4 (position 3). It is already in place, so no swap is needed: $[\mathbf{1, 2, 4}, 5, 8]$. (2 comparisons, 0 swaps)
Pass 4: minimum of $[5, 8]$ is 5 (position 4). Already in place: $[\mathbf{1, 2, 4, 5}, 8]$. (1 comparison, 0 swaps)
The last element is automatically in place. Totals: $4 + 3 + 2 + 1 = 10$ comparisons, only 2 real swaps.




Selection sort, pass 2: the minimum of the unsorted region (2) is located, then swapped with the first unsorted cell (5). The sorted region (green) grows by one element.

**Pseudocode of selection sort.**

```
FOR i ← 1 TO n - 1 DO
  minPos ← i
  FOR j ← i + 1 TO n DO
    IF A[j] < A[minPos] THEN minPos ← j
  END FOR
  IF minPos ≠ i THEN swap A[i] and A[minPos]
```

```
    END FOR
```

The inner loop only searches (it never swaps); the single swap happens after the inner loop, once the position of the minimum is known.

## 3.4 Efficiency: Comparisons and Swaps

Let us count precisely, for an array of $n$ elements. In **both** algorithms, pass $i$ performs $n - i$ comparisons, so the total number of comparisons is

$$(n-1) + (n-2) + \cdots + 2 + 1 = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}.$$

For $n = 5$: $\frac{5\times 4}{2} = 10$ comparisons — exactly what our two traces gave. For $n = 100$: 4 950 comparisons; for $n = 1\,000$: 499 500. The number of comparisons grows roughly like $n^2$: doubling the data quadruples the work.

**PROPERTY** **Complexity of bubble sort and selection sort**

Bubble sort and selection sort both perform $\frac{n(n-1)}{2}$ comparisons in the worst case: they are **quadratic** algorithms, written $O(n^2)$. They differ in the number of swaps:

- bubble sort may perform up to $\frac{n(n-1)}{2}$ swaps (one per comparison);
- selection sort performs at most $n - 1$ swaps (exactly one per pass).

(The proof is the summation above; being able to reproduce the count on a small array is examinable.)

**KEY POINTS** **Bubble sort vs selection sort**

| Criterion | Bubble sort | Selection sort |
|---|---|---|
| Idea | Swap adjacent out-of-order pairs | Find the minimum, swap it into place |
| Comparisons (worst case) | $\frac{n(n-1)}{2}$ | $\frac{n(n-1)}{2}$ |
| Swaps (worst case) | up to $\frac{n(n-1)}{2}$ | at most $n - 1$ |
| Early stop possible? | Yes (flag `swapped`) | No |
| Best when | Data nearly sorted | Swaps are costly (large records) |

**WORKED EXAMPLE 3** **GCE-style question**

The array $A = [7, 3, 9, 1]$ is to be sorted in ascending order.

1. Show the state of $A$ after one full pass of bubble sort.
2. Show the state of $A$ after one full pass of selection sort.
3. How many comparisons does each algorithm use in total on this array?

**SOLUTION**

1. Bubble sort, pass 1 (compare adjacent pairs, swap if out of order):

- 7 vs 3: swap → [3, 7, 9, 1]
- 7 vs 9: no swap → [3, 7, 9, 1]
- 9 vs 1: swap → [3, 7, 1, **9**]

State after pass 1: $\boxed{[3, 7, 1, 9]}$ — the largest value 9 has bubbled to the end.
2. Selection sort, pass 1: the minimum of [7, 3, 9, 1] is 1 (position 4); swap it with position 1: $\boxed{[1, 3, 9, 7]}$.

3. Comparisons: $n = 4$, so each algorithm uses $\frac{n(n-1)}{2} = \frac{4 \times 3}{2} = 6$ comparisons in total $(3 + 2 + 1)$.

**WARNING** **Exam tip — GCE**

A very common question is: "Give the content of the array after the second pass." Do not try to guess the final sorted array — the examiner wants the intermediate state. Apply the method rigorously, pass by pass, and mark the sorted region. Also remember: for bubble sort the sorted region grows at the right (largest values), for selection sort it grows at the left (smallest values).

**EXERCISE 3** **Practice**

1. Trace bubble sort on [6, 2, 5, 3, 9], giving the array after each pass and the total number of comparisons and swaps.
2. Trace selection sort on the same array. Compare the number of swaps with question 1 and comment.
3. A class contains 50 students. How many comparisons does selection sort need to rank their average marks? Use the formula $\frac{n(n-1)}{2}$.

4. Explain why the optimised bubble sort can be much faster than selection sort on the array $[1, 2, 3, 5, 4]$, even though both are $O(n^2)$.

**SOLUTION** **Exercise 1**

1. Bubble sort on $[6, 2, 5, 3, 9]$:
   - Pass 1: $6 \leftrightarrow 2$, $6 \leftrightarrow 5$, $6 \leftrightarrow 3$, $6 < 9 \rightarrow [2, 5, 3, 6, \mathbf{9}]$ (4 C, 3 S)
   - Pass 2: $2 < 5$, $5 \leftrightarrow 3$, $5 < 6 \rightarrow [2, 3, 5, \mathbf{6}, \mathbf{9}]$ (3 C, 1 S)
   - Pass 3: $2 < 3$, $3 < 5 \rightarrow [2, 3, \mathbf{5}, \mathbf{6}, \mathbf{9}]$ (2 C, 0 S)
   - Pass 4: $2 < 3 \rightarrow$ sorted (1 C). Total: 10 comparisons, 4 swaps.

2. Selection sort: min 2 swapped with 6 → $[2, 6, 5, 3, 9]$; min 3 swapped with 6 → $[2, 3, 5, 6, 9]$; min 5 in place; min 6 in place. 10 comparisons, 2 swaps — fewer swaps than bubble sort, as expected (at most $n - 1 = 4$).

3. $\frac{50 \times 49}{2} = 1225$ comparisons.

4. On $[1, 2, 3, 5, 4]$, one pass of bubble sort swaps 5 and 4; the next pass makes no swap, so the flag stops the algorithm after only $4 + 3 = 7$ comparisons. Selection sort cannot stop early: it always performs all $\frac{5 \times 4}{2} = 10$ comparisons.

**DID YOU KNOW?** **Did you know?**

Bubble sort is one of the oldest sorting algorithms ever described, and it is famously slow on large data — yet it survives in every syllabus because it is the simplest sort to understand and to trace by hand. In practice, real systems (banking software, the GCE Board's result processing) use much faster $O(n \log n)$ algorithms such as merge sort or quicksort, which you may meet in higher studies. But the ideas you have mastered here — comparisons, swaps, passes, sorted regions — are the building blocks of all of them.

You can now sort data in two different ways and, above all, justify the cost of each method. In the next section we complete the picture with **insertion sort** and learn how to choose the right algorithm for a given situation.

## 4 SORTING ALGORITHMS II: INSERTION SORT, COMPARISON AND CHOICE OF ALGORITHMS

In the previous section we studied **Bubble Sort** and **Selection Sort**. Both are simple to understand but share a major drawback: their time complexity is **$O(n^2)$** in almost every situation, even when the data is already nearly sorted. In this section we introduce **Insertion Sort**, an algorithm

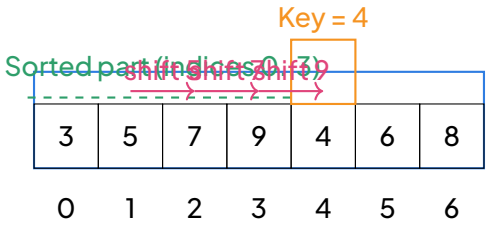
that keeps the $O(n^2)$ worst case but shines when the input is already partially ordered — a very common real-world scenario. We will then formalise **Big-O notation** to compare growth rates, build a complete comparison table of all algorithms seen so far, and give a practical decision guide for choosing the right tool.

## 4.1 Insertion Sort: Principle and Intuition

Imagine you are holding a hand of playing cards. You pick up the cards one by one from the table and insert each new card into its correct position among the cards you already hold, which are kept sorted. You do this by sliding larger cards one step to the right to make room. This is exactly how Insertion Sort works on an array.

**DEFINITION** **Insertion Sort**

Insertion Sort builds the final sorted array one element at a time. It maintains a **sorted subarray** at the left (indices 0 to i-1) and an **unsorted subarray** at the right (indices i to n-1). At each step i, it takes the element at index i (the **key**), compares it backwards through the sorted part, shifts all larger elements one position to the right, and inserts the key into the vacant spot.




Insertion Sort metaphor: the key (4) is inserted into the sorted left part by shifting larger elements right.

## 4.2 Step-by-Step Trace of Insertion Sort

Let us trace Insertion Sort on the array `A = [8, 3, 6, 2, 7]` (n=5). We count **comparisons** (key vs sorted elements) and **shifts** (moving an element one slot right).

**EXEMPLE** **Trace of Insertion Sort on [8, 3, 6, 2, 7]**

Initial: [8, 3, 6, 2, 7] Sorted part: [8] (index 0)
Pass i=1 (key=3):

- Compare 3 with 8 → 3 < 8, shift 8 right → [_, 8, 6, 2, 7] (1 comparison, 1 shift)

- Reached start, insert 3 at index 0 → [3, 8, 6, 2, 7]

Sorted part: [3, 8]
Pass i=2 (key=6):

- Compare 6 with 8 → 6 < 8, shift 8 right → [3, _, 8, 2, 7] (1 comp, 1 shift)
- Compare 6 with 3 → 6 > 3, stop. Insert 6 at index 1 → [3, 6, 8, 2, 7] (1 comp, 0 shift)

Sorted part: [3, 6, 8]
Pass i=3 (key=2):

- Compare 2 with 8 → shift 8 → [3, 6, _, 8, 7] (1 comp, 1 shift)
- Compare 2 with 6 → shift 6 → [3, _, 6, 8, 7] (1 comp, 1 shift)
- Compare 2 with 3 → shift 3 → [_, 3, 6, 8, 7] (1 comp, 1 shift)
- Reached start, insert 2 at index 0 → [2, 3, 6, 8, 7] (0 comp)

Sorted part: [2, 3, 6, 8]
Pass i=4 (key=7):

- Compare 7 with 8 → shift 8 → [2, 3, 6, _, 8] (1 comp, 1 shift)
- Compare 7 with 6 → 7 > 6, stop. Insert 7 at index 3 → [2, 3, 6, 7, 8] (1 comp, 0 shift)

Final sorted array: [2, 3, 6, 7, 8]
Summary counts:

| Pass (i) | Key | Comparisons | Shifts |
|---|---|---|---|
| 1 | 3 | 1 | 1 |
| 2 | 6 | 2 | 1 |
| 3 | 2 | 3 | 3 |
| 4 | 7 | 2 | 1 |
| Total | | 8 | 6 |

**WARNING** **Shifting vs Swapping**

In Bubble Sort and Selection Sort we **swap** two elements (three assignments). In Insertion Sort we **shift** elements one position right (one assignment per shift) and finally place the key (one assignment). A common mistake in traces is to write “swap key with 8” instead of “shift 8 right”. Remember: the key is held in a temporary variable; it is not swapped until the very end of the pass.

## 4.3 Pseudocode and Implementation

METHOD **Insertion Sort Algorithm**

1. For i from 1 to n-1:
2. key ← A[i]
3. j ← i - 1
4. While j ≥ 0 and A[j] > key:
5. A[j+1] ← A[j] (shift right)
6. j ← j - 1
7. End While
8. A[j+1] ← key (insert key)
9. End For

EXEMPLE **Python Implementation**

```
def insertion_sort(arr):
n = len(arr)
for i in range(1, n):
key = arr[i]
j = i - 1
# Shift elements greater than key to the right
while j >= 0 and arr[j] > key:
arr[j + 1] = arr[j]
j -= 1
arr[j + 1] = key
return arr
```

## 4.4 Efficiency of Insertion Sort

PROPERTY **Time Complexity of Insertion Sort**

- Worst case: $O(n^2)$ — when the array is reverse sorted. Every key must travel all the way to the front. Comparisons $\approx n^2/2$, Shifts $\approx n^2/2$.
- Average case: $O(n^2)$ — on random data, each key moves about halfway through the sorted part.

- Best case: O(n) — when the array is already sorted (or nearly sorted). The while loop condition fails immediately (0 shifts, 1 comparison per pass). Total comparisons = n−1.

Space complexity: O(1) auxiliary (in-place). Insertion Sort is **stable** (equal elements keep their relative order).

KEY POINTS **Why Insertion Sort matters**

Although $O(n^2)$ in the worst case, Insertion Sort is often the fastest of the simple sorts for:

- Small arrays ($n < 50$).
- Nearly sorted data (e.g. a list where only a few new entries are added).
- Online sorting (data arrives piece by piece).

Many hybrid algorithms (like Timsort used in Python and Java) switch to Insertion Sort for small subarrays.

## 4.5 Big-O Notation: Comparing Growth Rates

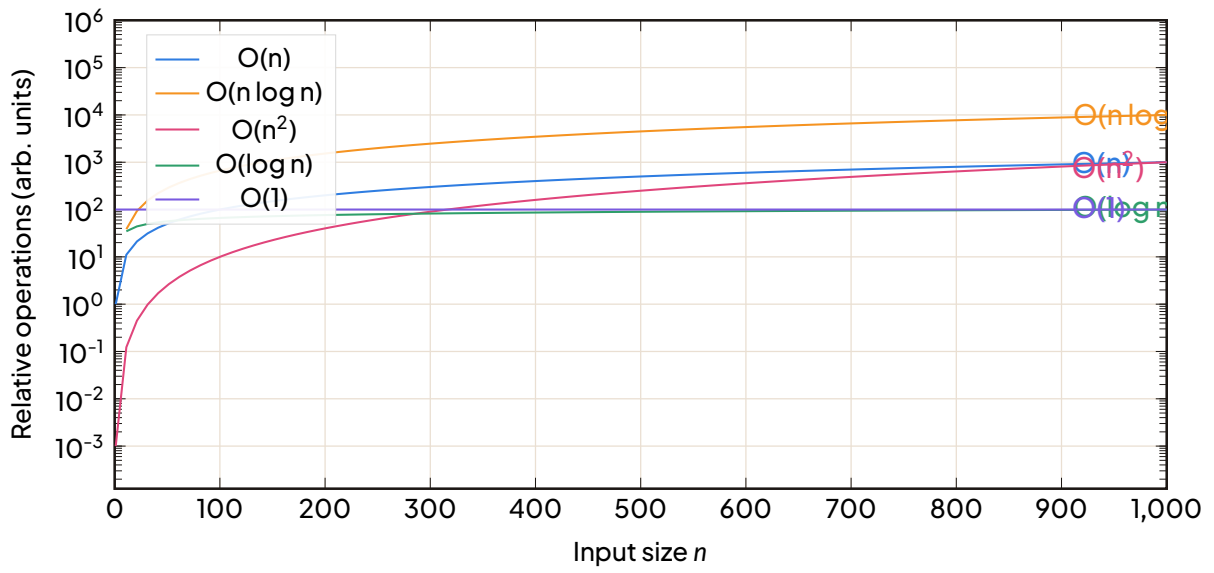
To compare algorithms independently of hardware, we use **Big-O notation**. It describes the asymptotic upper bound on the number of elementary operations as a function of input size n.

DEFINITION **Big-O Notation**

We say an algorithm runs in **O(f(n))** time if there exist constants $c > 0$ and $n_0$ such that for all $n \geq n_0$, the running time $T(n) \leq c \cdot f(n)$. In plain words: for large n, the running time grows no faster than a constant multiple of $f(n)$.

The common complexity classes we have met (and one we will meet later) are:

| Class | Name | Typical Algorithm |
|---|---|---|
| O(1) | Constant | Array access, hash lookup |
| O(log n) | Logarithmic | Binary Search |
| O(n) | Linear | Linear Search, Insertion Sort (best) |
| O(n log n) | Linearithmic | Merge Sort, Quicksort, Heapsort |
| $O(n^2)$ | Quadratic | Bubble, Selection, Insertion (worst/avg) |




Growth curves on a log-scale y-axis. Notice how $O(n^2)$ explodes while $O(n \log n)$ stays close to $O(n)$ for moderate n.

**DID YOU KNOW?** **Concrete timings**

Assume a computer does $10^9$ operations per second.

- n = 10,000: $O(n) \approx 0.01$ ms; $O(n \log n) \approx 0.13$ ms; $O(n^2) \approx 100$ seconds.
- n = 1,000,000: $O(n) \approx 1$ ms; $O(n \log n) \approx 20$ ms; $O(n^2) \approx 11.5$ days!

This is why $O(n \log n)$ sorts (Merge Sort, Quicksort) are used for large data in practice.

## 4.6 Complete Comparison Table

| Algorithm | Best | Average | Worst | Space | Stable? |
|---|---|---|---|---|---|
| Linear Search | $O(1)$ | $O(n)$ | $O(n)$ | $O(1)$ | N/A |
| Binary Search | $O(1)$ | $O(\log n)$ | $O(\log n)$ | $O(1)$ | N/A |
| Bubble Sort | $O(n)$ | $O(n^2)$ | $O(n^2)$ | $O(1)$ | Yes |
| Selection Sort | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ | $O(1)$ | No* |
| Insertion Sort | $O(n)$ | $O(n^2)$ | $O(n^2)$ | $O(1)$ | Yes |

* Selection Sort can be made stable with extra space or linked lists, but the standard array version is not stable.

KEY POINTS **Key takeaways from the table**

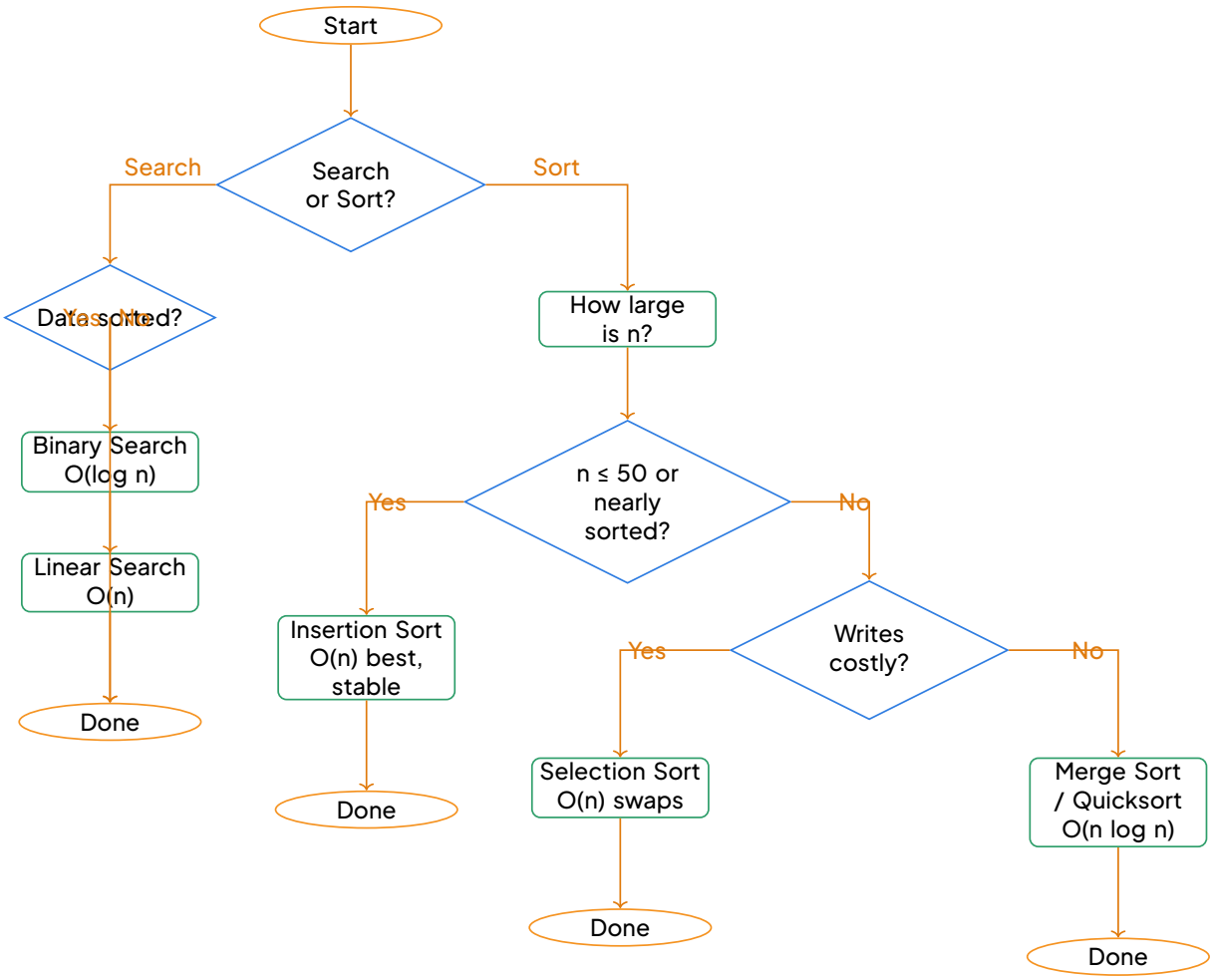
- Search: If data is sorted → Binary Search (O(log n)). If unsorted or tiny → Linear Search (O(n)).
- Sort small/nearly sorted: Insertion Sort (O(n) best, stable, in-place).
- Sort small, memory writes costly: Selection Sort (only O(n) swaps).
- Sort large data: Need O(n log n) algorithms (Merge Sort, Quicksort) — see enrichment box.

## 4.7 Decision Guide: Choosing the Right Algorithm

When you face a problem in the exam or in a project, follow this decision process.

METHOD **How to choose a searching/sorting algorithm**

1. What is the task? Search → go to step 2. Sort → go to step 3.
2. Search: Is the data sorted?
   - Yes → Binary Search (O(log n)).
   - No → Can you sort it once and search many times? If yes, sort first (step 3) then Binary Search. If only one search → Linear Search (O(n)).
3. Sort: How large is n?
   - $n \leq 50$ (or data nearly sorted) → Insertion Sort (simple, fast, stable).
   - n small but memory writes expensive (e.g. Flash/EEPROM) → Selection Sort (min swaps).
   - n large (hundreds or more) → Use an O(n log n) sort: Merge Sort (stable, guaranteed O(n log n)) or Quicksort (fast in practice, cache-friendly). In exams, if asked to "write an efficient sort for large n", name Merge Sort or Quicksort and state O(n log n).
4. Stability required? (Equal keys must keep original order) → Insertion Sort, Merge Sort, Bubble Sort. Avoid Selection Sort, Quicksort (standard version).
5. Extra memory allowed? O(1) only → Insertion, Selection, Bubble, Quicksort (in-place). O(n) allowed → Merge Sort (standard).




Decision flowchart for choosing a searching or sorting algorithm.

## 4.8 Worked Example: Algorithm Selection

**WORKED EXAMPLE 4** **Choosing algorithms for three scenarios**

For each scenario, state the most appropriate algorithm from the syllabus (Linear Search, Binary Search, Bubble Sort, Selection Sort, Insertion Sort) and justify your choice in terms of time complexity and data characteristics.

Scenario A: A school database of 800 students (sorted by admission number) must find a student by admission number. This search happens dozens of times per day.

Scenario B: A sensor records 20 temperature readings per minute into a small buffer. At the end of each minute, the buffer (size 20) must be sorted to find the median. The buffer is usually almost sorted because temperature changes slowly.

Scenario C: An embedded system with very limited RAM (2 KB) must sort an array of 100 integers stored in Flash memory. Writing to Flash is slow and wears out the memory.

SOLUTION

Solution:
A: Binary Search.

- Data is sorted (by admission number).
- Many repeated searches → O(log n) per search is crucial. 800 → $\log_2 800 \approx 10$ comparisons vs 400 average for Linear Search.
- No sorting needed.

B: Insertion Sort.

- n = 20 (small).
- Data is nearly sorted (temperature changes slowly) → Insertion Sort runs in near O(n) time (best case).
- Stable, in-place, simple to code.

C: Selection Sort.

- n = 100 (moderate, but $O(n^2) = 10{,}000$ ops is acceptable).
- Writes are costly (Flash wear). Selection Sort performs only O(n) swaps (writes), whereas Insertion Sort does $O(n^2)$ shifts (writes) in worst/average case.
- Space O(1) fits tight RAM.

## 4.9 Enrichment: Faster Sorting Algorithms (Beyond Syllabus)

DID YOU KNOW? **Merge Sort and Quicksort — O(n log n)**

The syllabus requires you to know that Merge Sort and Quicksort exist and achieve O(n log n) worst-case (Merge Sort) or average-case (Quicksort) time. You are not required to write their code or trace them in detail for the GCE, but knowing their names and complexity is an enrichment that shows depth.

- Merge Sort: Divide array in halves recursively, sort each half, then merge two sorted halves in O(n). Guaranteed O(n log n) worst case, stable, but needs O(n) extra space.
- Quicksort: Pick a pivot, partition array into elements < pivot and > pivot, recurse on both sides. In-place, cache-efficient, average O(n log n), but worst case $O(n^2)$ (rare with good pivot choice). Not stable.
- Built-in sorts: Python's `sorted()` / `list.sort()` (Timsort, hybrid Merge+Insertion), Java's `Arrays.sort()` (Dual-pivot Quicksort for primitives, Timsort for objects), C++ `std::sort` (Introsort = Quicksort + Heapsort fallback). Always prefer built-in sorts in real projects.

## 4.10 Practice Exercise

**EXERCISE 4** **Insertion Sort trace and algorithm choice**

1. Trace Insertion Sort on the array `B = [12, 5, 9, 5, 14, 3]`. Show the array after each complete pass (i = 1 to 5). Count total comparisons and total shifts.
2. Is Insertion Sort stable? Justify using your trace (note the two 5's).
3. A program must maintain a sorted list of at most 30 product codes. New codes are inserted one at a time, and the list must stay sorted after each insertion. Which sorting algorithm from the syllabus is most suitable? Explain.
4. A dataset of 50,000 records must be sorted once a week. The records are stored on disk. Which type of algorithm (name the complexity class and give an example) should be used? Why are the three algorithms studied in detail (Bubble, Selection, Insertion) unsuitable?

**SOLUTION** **Exercise: Insertion Sort trace and algorithm choice**

1. Trace on B = [12, 5, 9, 5, 14, 3]:
   - i=1, key=5: compare 5<12, shift 12 → [5, 12, 9, 5, 14, 3] (1 comp, 1 shift)
   - i=2, key=9: compare 9<12, shift 12; compare 9>5 stop → [5, 9, 12, 5, 14, 3] (2 comp, 1 shift)
   - i=3, key=5: compare 5<12 shift; 5<9 shift; 5=5? (5>5 false) stop at first 5 → [5, 5, 9, 12, 14, 3] (3 comp, 2 shifts) Stability preserved: new 5 placed after existing 5.
   - i=4, key=14: compare 14>12 stop → [5, 5, 9, 12, 14, 3] (1 comp, 0 shifts)
   - i=5, key=3: compare 3<14 shift; 3<12 shift; 3<9 shift; 3<5 shift; 3<5 shift; start → [3, 5, 5, 9, 12, 14] (5 comp, 5 shifts)

   Total comparisons: 1+2+3+1+5 = 12. Total shifts: 1+1+2+0+5 = 9.
2. Yes, Insertion Sort is stable. In pass i=3, the key 5 stopped at the first 5 (since 5 > 5 is false) and was inserted after it. The original order of the two 5's (index 0 then index 3) is preserved.
3. Insertion Sort. The list is small ($n \leq 30$) and we insert one element at a time into an already sorted list. This is exactly the best-case scenario for Insertion Sort: each insertion takes O(k) where k is current size, but since the new element is just one, it's essentially O(n) per insertion, or we can view the whole process as Insertion Sort's natural online behaviour. It is simple, in-place, and stable.
4. O(n log n) algorithm, e.g. Merge Sort or Quicksort. For n = 50,000, $O(n^2)$ algorithms

would require 2.5 billion operations — far too slow (minutes to hours). O(n log n) requires $50{,}000 \times 16 \approx 800{,}000$ operations — milliseconds. Bubble, Selection, Insertion are all $O(n^2)$ in worst/average case and cannot handle 50,000 records in reasonable time.

**KEY POINTS** **Chapter Summary: Searching & Sorting**

- Search: Linear (O(n), unsorted) vs Binary (O(log n), sorted).
- Simple Sorts ($O(n^2)$): Bubble (stable, detects sorted), Selection (min swaps), Insertion (fast on nearly sorted, stable, online).
- Big-O: Classifies growth: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2)$.
- Choose wisely: Consider n, sortedness, stability, memory, write cost.
- Real world: Use built-in O(n log n) sorts (Timsort, Introsort) for large data.

# 5 INTEGRATION ACTIVITY

## 5.1 Aims of the activity

KEY POINTS **Learning outcomes**

By the end of this integration activity, you should be able to:

- Trace and compare the behaviour of **selection sort** and **insertion sort** on a small data set, counting comparisons and data movements.
- Write clear pseudocode and implement a complete program (in C or Python) that sorts a list of student records by descending average and displays the top 10.
- Implement both **linear search** (on unsorted data) and **binary search** (on data sorted by the search key) to retrieve a student's rank and average.
- Analyse the time complexity of each algorithm and justify the choice of searching and sorting strategies for a growing database (from 60 to 6 000 records).
- Present and defend your algorithmic choices in a structured oral presentation.

## 5.2 Situation

EXPERIMENT **The College Results Board**

The principal of Government Bilingual High School (GBHS) Mfoundi in Yaoundé wishes to publish the Upper Sixth Honour Roll for the current academic year. The school has 60 students in Upper Sixth. Their names and annual averages (out of 20) are stored in two parallel arrays, `names[60]` and `averages[60]`, in the order of registration (i.e. unsorted).
The principal asks the Computer Science club to develop a program that will:

1. Sort the 60 students by descending average so that the top performers appear first.
2. Display the top 10 students (name, average, rank) on the school notice board.
3. Allow a user to type a student's name and instantly see that student's rank and average.

Because the club wants to learn and compare algorithms, they decide to:

- First work by hand on a sample of 8 students to trace two sorting algorithms (selection sort and insertion sort) and count their operations.
- Then write pseudocode and a full program for the complete list of 60.
- Implement two search modules: linear search on the original unsorted arrays, and bi-

nary search on a version of the arrays sorted by the search key.

- Compare the worst-case number of comparisons for 60 students and extrapolate to a regional database of 6 000 students (e.g. all Upper Sixth students in the Centre Region).

Sample data (8 students):

| Index | Name | Average |
|---|---|---|
| 0 | ABANDA Marc | 14.5 |
| 1 | BIKOI Alice | 16.2 |
| 2 | CHOUAIBOU Karim | 12.8 |
| 3 | DJOUMESSI Fatima | 18.0 |
| 4 | ESSOMBA Paul | 15.5 |
| 5 | FOUDA Grace | 13.0 |
| 6 | GUIMDO Serge | 17.3 |
| 7 | HAPPI Mireille | 11.9 |

The club will work in groups of 3–4. Each group must produce:

- Hand-written traces for both sorts on the 8-student sample.
- Pseudocode and a working C or Python program for the full 60-student list.
- A short report (≈300 words) comparing the search strategies and recommending one for the regional scale.
- A 10-minute oral defence of their work.

## 5.3 Tasks

1. **Hand trace of sorting algorithms (sample of 8).**
   (a) Trace **selection sort** (descending order) on the `averages` array, keeping `names` in parallel. Show the array after each outer iteration. Count the total number of comparisons between averages and the total number of swaps (exchanges of both average and name).
   (b) Trace **insertion sort** (descending order) on the same initial data. Show the array after each insertion step. Count the total number of comparisons and shifts (moving an element one position to the right).
   (c) Which algorithm performed fewer comparisons on this sample? Which performed fewer data moves? Write a one-sentence conclusion.
2. **Pseudocode and program for the full list (60 students).**
   (a) Write pseudocode for a **selection sort** that sorts two parallel arrays (`names`, `averages`) in descending order of `averages`.

(b) Write pseudocode for a function `displayTop10(names, averages)` that prints the rank (1–10), name, and average of the first ten entries.

(c) Implement the complete program in **C** or **Python**. The program must:

- Read the 60 names and averages from a text file `results.txt` (one line per student: `Name Average`).
- Sort the arrays using your selection sort.
- Call `displayTop10`.
- Enter a loop: prompt the user for a name; if the name is `"END"` stop; otherwise search for the name and display rank and average (see Task 3).

3. **Search modules.**

(a) Write pseudocode for `linearSearch(names, n, target)` that returns the index of `target` in the **original unsorted** `names` array (or -1 if not found). Count the number of comparisons made.

(b) Write pseudocode for `binarySearch(sortedNames, sortedAverages, n, target)` that assumes the arrays are **sorted alphabetically by name** (the search key). The function must return the index and the average. Count the number of comparisons.

(c) In your program, implement both searches. For a given query, first run linear search on the original arrays (you must keep a copy), then run binary search on the name-sorted copy. Display the number of comparisons each made.

4. **Complexity analysis and justification.**

(a) Give the worst-case number of comparisons for linear search and binary search when $n = 60$ and when $n = 6\,000$.

(b) Assuming the honour roll is generated once per term but the search module is used **daily** by hundreds of students, which search strategy should the school adopt? Justify in a paragraph (≈100 words) considering both time complexity and the overhead of keeping an array sorted.

(c) If the regional database grows to 6 000 records and new students are added frequently, which sorting algorithm (selection sort or insertion sort) would be more suitable for incremental updates? Explain briefly.

## 5.4 Model answer

**WARNING** **A crucial precondition for binary search — read before tracing**

Binary search only works on an array that is sorted by the search key. Here the user types a name, so binary search requires an array sorted alphabetically by name. The honour-roll array is sorted by average, which does not order the names alphabetically; applying binary search by name on it would give wrong results. The correct design is to keep two sorted copies:

- one copy sorted by descending average (for the honour roll and the rank);

- one copy sorted alphabetically by name (for binary search).

Linear search, by contrast, has no precondition: it works on the original unsorted arrays. This distinction is exactly what the GCE examiner expects you to state.

**EXEMPLE** **Task 1: Hand traces on the 8-student sample**

Initial averages array:

| Index | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| Average | 14.5 | 16.2 | 12.8 | 18.0 | 15.5 | 13.0 | 17.3 | 11.9 |

Selection sort (descending). Outer loop $i = 0$ to $n - 2$: find the index `maxIdx` of the maximum in positions $i$ to $n - 1$, then swap it into position $i$ (the `names` array is swapped in parallel). Bold entries are those placed (or confirmed) during the pass.

| Pass | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| Start | 14.5 | 16.2 | 12.8 | 18.0 | 15.5 | 13.0 | 17.3 | 11.9 |
| $i = 0$ | 18.0 | 16.2 | 12.8 | 14.5 | 15.5 | 13.0 | 17.3 | 11.9 |
| $i = 1$ | 18.0 | 17.3 | 12.8 | 14.5 | 15.5 | 13.0 | 16.2 | 11.9 |
| $i = 2$ | 18.0 | 17.3 | 16.2 | 14.5 | 15.5 | 13.0 | 12.8 | 11.9 |
| $i = 3$ | 18.0 | 17.3 | 16.2 | 15.5 | 14.5 | 13.0 | 12.8 | 11.9 |
| $i = 4$ | 18.0 | 17.3 | 16.2 | 15.5 | 14.5 | 13.0 | 12.8 | 11.9 |
| $i = 5$ | 18.0 | 17.3 | 16.2 | 15.5 | 14.5 | 13.0 | 12.8 | 11.9 |
| $i = 6$ | 18.0 | 17.3 | 16.2 | 15.5 | 14.5 | 13.0 | 12.8 | 11.9 |

Pass-by-pass accounting:

- $i = 0$: scan indices 0–7, max = 18.0 at index 3; swap indices 0 and 3. Comparisons: 7, swaps: 1.
- $i = 1$: scan indices 1–7, max = 17.3 at index 6; swap indices 1 and 6. Comparisons: 6, swaps: 1.
- $i = 2$: scan indices 2–7, max = 16.2 at index 6; swap indices 2 and 6. Comparisons: 5, swaps: 1.
- $i = 3$: scan indices 3–7, max = 15.5 at index 4; swap indices 3 and 4. Comparisons: 4, swaps: 1.
- $i = 4$: scan indices 4–7, max = 14.5 already at index 4; no swap needed. Comparisons: 3, swaps: 0.
- $i = 5$: scan indices 5–7, max = 13.0 already at index 5; no swap. Comparisons: 2, swaps: 0.

- $i = 6$: scan indices 6–7, max = 12.8 already at index 6; no swap. Comparisons: 1, swaps: 0.

Total comparisons = $7 + 6 + 5 + 4 + 3 + 2 + 1 = \frac{n(n-1)}{2} = 28$ (always, whatever the data). Total real swaps = 4 (at most one per pass, so at most $n - 1$).

Insertion sort (descending). Outer loop $i = 1$ to $n - 1$: take `key = averages[i]` (and the parallel name), shift every element of the sorted prefix that is smaller than the key one position to the right, then insert the key in the gap.

| Pass | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| Start | 14.5 | 16.2 | 12.8 | 18.0 | 15.5 | 13.0 | 17.3 | 11.9 |
| $i = 1$ | 16.2 | 14.5 | 12.8 | 18.0 | 15.5 | 13.0 | 17.3 | 11.9 |
| $i = 2$ | 16.2 | 14.5 | 12.8 | 18.0 | 15.5 | 13.0 | 17.3 | 11.9 |
| $i = 3$ | 18.0 | 16.2 | 14.5 | 12.8 | 15.5 | 13.0 | 17.3 | 11.9 |
| $i = 4$ | 18.0 | 16.2 | 15.5 | 14.5 | 12.8 | 13.0 | 17.3 | 11.9 |
| $i = 5$ | 18.0 | 16.2 | 15.5 | 14.5 | 13.0 | 12.8 | 17.3 | 11.9 |
| $i = 6$ | 18.0 | 17.3 | 16.2 | 15.5 | 14.5 | 13.0 | 12.8 | 11.9 |
| $i = 7$ | 18.0 | 17.3 | 16.2 | 15.5 | 14.5 | 13.0 | 12.8 | 11.9 |

Comparisons and shifts per pass (we compare the key with each prefix element from right to left; each smaller element found is shifted):

- $i = 1$: key = 16.2; compare with 14.5 (smaller, shift); prefix exhausted. 1 comparison, 1 shift.
- $i = 2$: key = 12.8; compare with 14.5 (larger, stop). 1 comparison, 0 shifts.
- $i = 3$: key = 18.0; compare with 12.8, 14.5, 16.2 (all smaller, all shifted); prefix exhausted. 3 comparisons, 3 shifts.
- $i = 4$: key = 15.5; compare with 12.8 (shift), 14.5 (shift), 16.2 (larger, stop). 3 comparisons, 2 shifts.
- $i = 5$: key = 13.0; compare with 12.8 (shift), 14.5 (shift), 15.5 (shift), 16.2 (larger, stop). 4 comparisons, 3 shifts.
- $i = 6$: key = 17.3; compare with 12.8, 13.0, 14.5, 15.5, 16.2 (all smaller, all shifted), then 18.0 (larger, stop). 6 comparisons, 5 shifts. The key is inserted at index 1, not index 0, because 17.3 < 18.0.
- $i = 7$: key = 11.9; compare with 12.8 (larger, stop). 1 comparison, 0 shifts.

Total comparisons = $1 + 1 + 3 + 3 + 4 + 6 + 1 = 19$. Total shifts = $1 + 0 + 3 + 2 + 3 + 5 + 0 = 14$.

Conclusion: on this sample, insertion sort used fewer comparisons (19 against 28) because it stops scanning as soon as the right position is found, whereas selection sort always scans the whole unsorted region; the data movements are comparable (14 shifts

against 4 swaps, each swap costing 3 assignments per array). Insertion sort adapts to partially ordered data; selection sort always performs exactly $\frac{n(n-1)}{2}$ comparisons.

METHOD **Task 2: Pseudocode for selection sort and displayTop10**

Selection sort (parallel arrays, descending):

```
procedure selectionSortDesc(names, averages, n)
    for i = 0 to n-2 do
        maxIdx = i
        for j = i+1 to n-1 do
            if averages[j] > averages[maxIdx] then
                maxIdx = j
            end if
        end for
        if maxIdx <> i then
            swap(averages[i], averages[maxIdx])
            swap(names[i], names[maxIdx])
        end if
    end for
end procedure
```

Display top 10:

```
procedure displayTop10(names, averages)
    print "Rank | Name | Average"
    for i = 0 to 9 do
        print i+1, names[i], averages[i]
    end for
end procedure
```

EXEMPLE **Task 2: Python implementation (complete program)**

Structure of the program `honours.py` (presented as annotated pseudocode close to Python):

```
def selection_sort_desc(names, averages):
    n = len(averages)
    for i in range(n - 1):
        max_idx = i
        for j in range(i + 1, n):
            if averages[j] > averages[max_idx]: max_idx = j
        if max_idx != i:
            swap averages[i] with averages[max_idx]
            swap names[i] with names[max_idx]
def display_top10(names, averages):
    print header "Rank Name Average"
    for i in range(10): print i+1, names[i], averages[i]
def read_file(filename):
    open file; for each line, split into name and average;
    append name to names list, float(average) to averages list
    return names, averages
```

The main part reads `results.txt`, keeps an original copy for linear search, sorts a second copy by descending average for the honour roll, sorts a third copy alphabetically by name for binary search (Task 3), displays the top 10, then enters the query loop.

EXEMPLE **Task 3: Search pseudocode and Python functions**

Linear search (on the original unsorted arrays):

```
function linearSearch(origNames, n, target)
    comparisons = 0
    for i = 0 to n-1 do
        comparisons = comparisons + 1
```

```
        if origNames[i] = target then
            return i, comparisons
        end if
    end for
    return -1, comparisons
end function
```

Binary search (on the copy sorted alphabetically by name):

```
function binarySearch(sortedNames, sortedAverages, n, target)
    low = 0; high = n-1; comparisons = 0
    while low <= high do
        mid = (low + high) div 2
        comparisons = comparisons + 1
        if sortedNames[mid] = target then
            return mid, comparisons
        else if sortedNames[mid] < target then
            low = mid + 1
        else
            high = mid - 1
        end if
    end while
    return -1, comparisons
end function
```

Main query loop (Python-style):

```
orig_names, orig_averages = read_file("results.txt")
avg_names, avg_averages = copies sorted by descending average
display_top10(avg_names, avg_averages)
name_names, name_averages = copies sorted alphabetically by name
repeat:
    query = input("Enter student name (END to quit): ")
```

```
    if query = "END": stop
    idxL, compL = linear_search(orig_names, query)
    idxB, compB = binary_search(name_names, name_averages, query)
    if found: display rank and average from each method,
    plus the comparison counts compL and compB
```

Note: the rank on the honour roll is the position in the average-sorted array (index +1); the binary search gives the position in the name-sorted array, from which we retrieve the average, and the rank can then be recovered by locating that average in the average-sorted array.

**EXEMPLE** **Task 4: Complexity analysis and justification**

Worst-case number of comparisons:

| $n$ | Linear search (worst case) | Binary search (worst case) |
|---|---|---|
| 60 | 60 | $\lceil \log_2 60 \rceil = 6$ |
| 6 000 | 6 000 | $\lceil \log_2 6000 \rceil = 13$ |

(For binary search the worst case is $\lceil \log_2 n \rceil$ or $\lfloor \log_2 n \rfloor + 1$ depending on convention; with $n = 60$, $2^5 = 32 < 60 \leq 64 = 2^6$, so 6 comparisons suffice; with $n = 6\,000$, $2^{12} = 4\,096 < 6\,000 \leq 8\,192 = 2^{13}$, so 13 comparisons suffice.)

Recommendation paragraph (≈100 words): For the daily search module, the school should adopt binary search on an alphabetically sorted copy of the student list. With 60 students, linear search costs up to 60 comparisons per query while binary search needs at most 6 — already a gain, but modest. At the regional scale of 6 000 students, the gap becomes decisive: 6 000 comparisons against only 13. Since the honour roll is produced once per term, the one-time cost of sorting a copy by name is negligible compared with hundreds of daily queries. Keeping two sorted copies (by average for the roll, by name for searching) is simple and gives optimal performance for both tasks. Binary search is therefore the clear choice for a growing database.

Incremental updates: if new students are added frequently, insertion sort is more suitable than selection sort for maintaining the sorted arrays. Inserting one new record into an already sorted array costs only $O(n)$ (find the position, shift, insert), and insertion sort runs in $O(n)$ on nearly sorted data, whereas selection sort always costs $O(n^2)$ comparisons regardless of the existing order. For frequent incremental updates, insertion sort (or binary insertion sort, which locates the position by binary search) should be preferred.

KEY POINTS **Key takeaways for the defence**

- Selection sort: exactly $\frac{n(n-1)}{2}$ comparisons in all cases, at most $n - 1$ swaps; simple but not adaptive.
- Insertion sort: $O(n^2)$ in the worst case but $O(n)$ on nearly sorted data; adaptive, and performs fewer comparisons on partially ordered sets.
- Linear search: $O(n)$, no precondition on the data, but slow for large $n$.
- Binary search: $O(\log n)$, but requires the array to be sorted by the search key; keep one sorted copy per search key.
- For a static honour roll (sorted once per term) with frequent name searches, keep two sorted arrays — one by average, one by name — and use binary search on the name-sorted array.
- For incremental updates, insertion sort outperforms selection sort because it exploits the existing order.

# 6 METHODS AND WORKED EXAMPLES

# 7 CHAPTER CSC03: APPLYING SEARCHING AND SORTING TECHNIQUES

## 7.1 Searching Algorithms I: Linear Search

**METHOD** **Linear Search Algorithm**

Goal: Find the index of a target value in an unsorted array (or confirm its absence).

1. Input: Array $A$ of size $n$, target value $key$.
2. Initialise: Set $i \leftarrow 0$ (first index).
3. Loop: While $i < n$:
   - If $A[i] = key$, return $i$ (found).
   - Else $i \leftarrow i + 1$.
4. Not found: After loop, return -1 (or a sentinel value).

Key Reflexes:

- Works on any array (sorted or unsorted).
- Time complexity: Best $O(1)$, Worst $O(n)$, Average $O(n)$.
- Space complexity: $O(1)$ (in-place, iterative).

**WORKED EXAMPLE 5** **Linear Search Trace and Complexity**

Consider the array $A = [12, 7, 25, 3, 18, 7, 30]$ (0-indexed).

1. Trace the linear search algorithm to find the first occurrence of $key = 7$. Show the value of $i$ and $A[i]$ at each iteration.
2. How many comparisons are made in the best case for this specific array? In the worst case?
3. Write the pseudocode for a version that returns all indices where $key$ occurs.

**SOLUTION**

Detailed Solution

1. Trace for $key = 7$ (first occurrence):

| Iteration | $i$ | $A[i]$ | $A[i] = 7$? |
|---|---|---|---|
| 1 | 0 | 12 | No |
| 2 | 1 | 7 | Yes → Return 1 |

   The algorithm stops at iteration 2 and returns index 1.

2. Comparisons:
   - Best case: $key = 12$ (first element). Only 1 comparison.
   - Worst case: $key = 30$ (last element) or $key = 100$ (absent). 7 comparisons ($n$ comparisons).

3. Pseudocode for all indices:

```
Function LinearSearchAll(A, n, key)
  indices ← empty list
  For i ← 0 to n-1
    If A[i] = key Then
      Append i to indices
    End If
  End For
  Return indices
End Function
```

## 7.2 Searching Algorithms II: Binary Search

METHOD **Binary Search Algorithm (Iterative)**

Precondition: Array $A$ must be sorted in ascending order. Goal: Find index of $key$ in $O(\log n)$ time.

1. Initialise: $low \leftarrow 0$, $high \leftarrow n - 1$.
2. Loop: While $low \leq high$:
   - $mid \leftarrow \lfloor (low + high)/2 \rfloor$ (integer division).
   - If $A[mid] = key$: Return $mid$.

- Else if $A[mid] < key$: $low \leftarrow mid + 1$ (search right half).
- Else: $high \leftarrow mid - 1$ (search left half).

3. Not found: Return -1.

Key Reflexes:

- Mid calculation: Use $low + \lfloor (high - low)/2 \rfloor$ to avoid integer overflow (good practice).
- Loop condition: $low \leq high$ (not <). If $low = high$, one element remains to check.
- Update bounds: Always $mid \pm 1$ to avoid infinite loops.
- Complexity: Time $O(\log_2 n)$, Space $O(1)$ iterative / $O(\log n)$ recursive (stack).

**WORKED EXAMPLE 6** **Binary Search Trace and Boundary Analysis**

Given the sorted array $A = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]$ (size $n = 10$, indices 0..9).

1. Trace the iterative binary search for $key = 23$. Draw a table with columns: Iteration, $low$, $high$, $mid$, $A[mid]$, Decision.
2. Trace for $key = 50$ (not in array). Show the final values of $low$ and $high$ when the loop terminates.
3. Explain why the condition is $low \leq high$ and not $low < high$. Give a concrete example from the array above where $low < high$ would fail.

**SOLUTION**

Detailed Solution

1. Trace for $key = 23$:

| Iter | $low$ | $high$ | $mid$ | $A[mid]$ | Decision |
|---|---|---|---|---|---|
| 1 | 0 | 9 | 4 | 16 | $16 < 23 \rightarrow low = 5$ |
| 2 | 5 | 9 | 7 | 56 | $56 > 23 \rightarrow high = 6$ |
| 3 | 5 | 6 | 5 | 23 | Found! Return 5 |

2. Trace for $key = 50$:

| Iter | $low$ | $high$ | $mid$ | $A[mid]$ | Decision |
|---|---|---|---|---|---|
| 1 | 0 | 9 | 4 | 16 | $16 < 50 \rightarrow low = 5$ |
| 2 | 5 | 9 | 7 | 56 | $56 > 50 \rightarrow high = 6$ |
| 3 | 5 | 6 | 5 | 23 | $23 < 50 \rightarrow low = 6$ |
| 4 | 6 | 6 | 6 | 38 | $38 < 50 \rightarrow low = 7$ |

Loop condition $low \leq high$ becomes $7 \leq 6$ (False). Terminates. Returns -1. Final state: $low = 7, high = 6$. Note: $low$ is the insertion point (index of first element > 50).

3. Why $low \leq high$? If $low = high$, there is exactly one element left to check (the search space has size 1). Example: Search for $key = 91$ (last element).
   - Iter 1: $low = 0, high = 9, mid = 4, A[4] = 16 \rightarrow low = 5$.
   - Iter 2: $low = 5, high = 9, mid = 7, A[7] = 56 \rightarrow low = 8$.
   - Iter 3: $low = 8, high = 9, mid = 8, A[8] = 72 \rightarrow low = 9$.
   - Iter 4: $low = 9, high = 9$. With $\leq$: Loop enters. $mid = 9, A[9] = 91$. Found!
   - With $<$: Condition $9 < 9$ is False. Loop skips. Returns -1 (Missed the last element!).

## 7.3 Sorting Algorithms I: Bubble Sort and Selection Sort

**METHOD** **Bubble Sort Algorithm**

Goal: Sort array $A$ of size $n$ in ascending order (in-place). Idea: Repeatedly step through the list, compare adjacent elements and swap if in wrong order. Largest elements "bubble" to the end.

1. Outer loop (Passes): For $pass \leftarrow 0$ to $n - 2$ (or $n - 1$).
2. Inner loop (Comparisons): For $i \leftarrow 0$ to $n - 2 - pass$.
   - If $A[i] > A[i + 1]$: Swap $A[i]$ and $A[i + 1]$.
3. Optimisation (Early Exit): If a complete pass makes zero swaps, the array is sorted → Break.

Key Reflexes:

- Inner loop bound shrinks by 1 each pass (last $pass$ elements are already sorted).
- Stability: Stable (equal elements never swap).
- Complexity: Time Worst/Average $O(n^2)$, Best $O(n)$ (with early exit on sorted array). Space $O(1)$.

**WORKED EXAMPLE 7** **Bubble Sort Trace with Early Exit**

Sort the array $A = [5, 1, 4, 2, 8]$ using Bubble Sort with the early exit optimisation.

1. Show the state of the array after each complete pass. Indicate the number of swaps in that pass.

2. How many passes are actually executed? How many comparisons in total?
3. If the array was initially sorted [1, 2, 4, 5, 8], how many passes and comparisons would occur?

---

**SOLUTION**

Detailed Solution

1. Trace: $n = 5$. Indices 0..4.
   - Pass 0 ($i$ = 0..3): Compare (5,1) → Swap → [1, 5, 4, 2, 8] Compare (5,4) → Swap → [1, 4, 5, 2, 8] Compare (5,2) → Swap → [1, 4, 2, 5, 8] Compare (5,8) → No Swap. Array: [1, 4, 2, 5, 8], Swaps: 3
   - Pass 1 ($i$ = 0..2): Compare (1,4) → No Swap. Compare (4,2) → Swap → [1, 2, 4, 5, 8] Compare (4,5) → No Swap. Array: [1, 2, 4, 5, 8], Swaps: 1
   - Pass 2 ($i$ = 0..1): Compare (1,2) → No Swap. Compare (2,4) → No Swap. Array: [1, 2, 4, 5, 8], Swaps: 0 → STOP (Early Exit)
2. Executed Passes: 3 (Pass 0, 1, 2). Pass 3 and 4 skipped. Comparisons: Pass 0: 4, Pass 1: 3, Pass 2: 2. Total = 9 comparisons.
3. Already sorted array [1, 2, 4, 5, 8]: Pass 0: 4 comparisons, 0 swaps → Early Exit immediately. Passes: 1, Comparisons: 4 ($n - 1$). Best case $O(n)$ achieved.

**METHOD** **Selection Sort Algorithm**

Goal: Sort array $A$ of size $n$ in ascending order (in-place). Idea: Divide array into sorted prefix (left) and unsorted suffix (right). Repeatedly select the minimum from unsorted part and swap it with the first element of unsorted part.

1. For $i \leftarrow 0$ to $n - 2$ (boundary of sorted part):
   - $min_idx \leftarrow i$.
   - For $j \leftarrow i + 1$ to $n - 1$:
   - If $A[j] < A[min_idx]$: $min_idx \leftarrow j$.
   - If $min_idx \neq i$: Swap $A[i]$ and $A[min_idx]$.

Key Reflexes:

- Exactly $n-1$ swaps maximum (one per pass). Minimises data movement (good for EEPROM/Flash).
- Not Stable: Swapping $A[i]$ with a distant $A[min_idx]$ can move equal elements past each other.
- Complexity: Time Always $O(n^2)$ (Best/Average/Worst). No early exit. Space $O(1)$.

**WORKED EXAMPLE 8** **Selection Sort Trace and Stability Counter-Example**

1. Trace Selection Sort on $A = [29, 10, 14, 37, 13]$. Show the array after each pass $i$, the $min_idx$ found, and whether a swap occurred.
2. Demonstrate that Selection Sort is unstable using the array $B = [5_a, 5_b, 3]$ where $5_a$ and $5_b$ are equal keys with distinct identities (e.g., records). Show the final order of the 5s.
3. Why does Selection Sort have the same $O(n^2)$ time complexity even on an already sorted array?

**SOLUTION**

Detailed Solution

1. Trace on $[29, 10, 14, 37, 13]$:

| Pass $i$ | Unsorted Part | $min_idx$ | $min_val$ | Array After Pass |
|---|---|---|---|---|
| 0 | $[29, 10, 14, 37, 13]$ | 1 | 10 | Swap 0,1 → $[10, 29, 14, 37, 13]$ |
| 1 | $[29, 14, 37, 13]$ | 4 | 13 | Swap 1,4 → $[10, 13, 14, 37, 29]$ |
| 2 | $[14, 37, 29]$ | 2 | 14 | No Swap ($min_idx = 2 = i$) → $[10, 13, 14, 37, 29]$ |
| 3 | $[37, 29]$ | 4 | 29 | Swap 3,4 → $[10, 13, 14, 29, 37]$ |

Sorted array: $[10, 13, 14, 29, 37]$.

2. Stability Counter-Example: $B = [5_a, 5_b, 3]$ (indices 0,1,2).
   - Pass $i = 0$: Unsorted $[5_a, 5_b, 3]$. Min is 3 at index 2.
   - Swap $B[0]$ ($5_a$) with $B[2]$ (3).
   - Array becomes: $[3, 5_b, 5_a]$.

   Original order of 5s: $5_a$ then $5_b$. Final order: $5_b$ then $5_a$. Order reversed → Unstable.
3. Why always $O(n^2)$? The inner loop always scans the entire unsorted portion ($n - i - 1$ elements) to find the minimum, regardless of the initial order. It cannot "detect" sortedness. Comparisons $\sum_{i=0}^{n-2}(n - i - 1) = n(n-1)/2 \in \Theta(n^2)$.

## 7.4 Sorting Algorithms II: Insertion Sort, Comparison and Choice of Algorithms

**METHOD** **Insertion Sort Algorithm**

Goal: Sort array $A$ of size $n$ in ascending order (in-place). Idea: Build sorted prefix one el-

ement at a time. Take next element ($key$), shift larger elements right to make a hole, insert $key$.

1. For $i \leftarrow 1$ to $n - 1$:
   - $key \leftarrow A[i]$.
   - $j \leftarrow i - 1$.
   - While $j \geq 0$ AND $A[j] > key$:
   - $A[j + 1] \leftarrow A[j]$ (Shift right).
   - $j \leftarrow j - 1$.
   - $A[j + 1] \leftarrow key$ (Insert).

Key Reflexes:

- Stable: Condition is $A[j] > key$ (strict), so equal elements are not shifted past $key$.
- Adaptive: Best case $O(n)$ (already sorted: inner loop never runs). Worst case $O(n^2)$ (reverse sorted).
- Online: Can sort a list as it arrives (streaming).
- Low Overhead: Excellent for small $n$ ($n < 50$) or nearly sorted data. Often used as base case in hybrid sorts (Timsort, Introsort).

**WORKED EXAMPLE 9** **Insertion Sort Trace and Shift Counting**

Sort $A = [12, 11, 13, 5, 6]$ using Insertion Sort.

1. Trace the algorithm. For each $i = 1..4$, show: $key$, the shifting process (values of $j$ and $A[j]$ shifted), and the array after insertion.
2. Count the total number of shifts (assignments $A[j + 1] \leftarrow A[j]$) and comparisons ($A[j] > key$ evaluated).
3. If the array was reverse sorted $[13, 12, 11, 6, 5]$, what would be the total shifts and comparisons? Relate to formula.

**SOLUTION**

Detailed Solution

1. Trace:
   - i=1: $key = 11$. $j = 0, A[0] = 12 > 11$. Shift $A[1] = 12$. $j = -1$. Insert $A[0] = 11$. Array: $[11, 12, 13, 5, 6]$. (1 shift, 1 comp + 1 fail).
   - i=2: $key = 13$. $j = 1, A[1] = 12 \not> 13$. No shift. Insert $A[2] = 13$. Array: $[11, 12, 13, 5, 6]$. (0

shifts, 1 comp).

- i=3: $key = 5$. $j = 2, A[2] = 13 > 5$ Shift → $A[3] = 13$. $j = 1, A[1] = 12 > 5$ Shift → $A[2] = 12$. $j = 0, A[0] = 11 > 5$ Shift → $A[1] = 11$. $j = -1$. Insert $A[0] = 5$. Array: [5, 11, 12, 13, 6]. (3 shifts, 3 comps + 1 fail).
- i=4: $key = 6$. $j = 3, A[3] = 13 > 6$ Shift → $A[4] = 13$. $j = 2, A[2] = 12 > 6$ Shift → $A[3] = 12$. $j = 1, A[1] = 11 > 6$ Shift → $A[2] = 11$. $j = 0, A[0] = 5 \not> 6$. Insert $A[1] = 6$. Array: [5, 6, 11, 12, 13]. (3 shifts, 4 comps).

2. Totals: Shifts = 1 + 0 + 3 + 3 = **7**. Comparisons = (1 + 1) + (1) + (3 + 1) + (3 + 1) = **11** (counting the failing comparison each while loop).
3. Reverse Sorted [13, 12, 11, 6, 5] (Worst Case): For $i = 1..4$, $key$ is smaller than all previous. Inner loop runs $i$ times. Shifts = $\sum_{i=1}^{4} i = 10 = n(n-1)/2$. Comparisons = $\sum_{i=1}^{4}(i+1) = (2 + 3 + 4 + 5) = 14 = n(n-1)/2 + (n-1)$. Time complexity $\Theta(n^2)$.

**METHOD** **Choosing a Sorting Algorithm: Decision Framework**

No single "best" sort. Choice depends on constraints. Use this checklist:

1. Data Size ($n$):
   - Small ($n \leq 50$): Insertion Sort (low overhead, fast in practice).
   - Large: Need $O(n \log n)$ (Merge, Quick, Heap, Timsort).
2. Initial Order:
   - Nearly Sorted: Insertion Sort ($O(n)$), Timsort (built for this).
   - Random: Quick Sort (average $O(n \log n)$, cache friendly), Merge Sort (guaranteed $O(n \log n)$).
   - Reverse Sorted: Avoid Quick Sort (worst $O(n^2)$ without good pivot). Use Merge/Heap.
3. Stability Required? (Preserve order of equal keys)
   - Yes: Merge Sort, Insertion Sort, Bubble Sort, Timsort, Counting/Radix.
   - No: Quick Sort, Heap Sort, Selection Sort.
4. Memory Constraints:
   - $O(1)$ Extra Space (In-place): Quick Sort (stack $\log n$), Heap Sort, Insertion/Selection/Bubble.
   - $O(n)$ Auxiliary Array OK: Merge Sort (standard), Timsort.
5. Data Type / Keys:

- Integers in small range [0..$k$]: Counting Sort $O(n + k)$.
- Integers / Strings (fixed length): Radix Sort $O(d(n + b))$.

**WORKED EXAMPLE 10** **Algorithm Selection for Cameroonian Context Scenarios**

For each scenario, state the most appropriate sorting algorithm from the syllabus (Bubble, Selection, Insertion, Merge, Quick) and justify using at least two criteria from the decision framework.

1. A microcontroller in a Yaoundé traffic light system sorts 20 sensor readings (integers 0–100) every second. RAM is 2 KB. The array is often nearly sorted.
2. A database server in Douala sorts 1,000,000 customer records by "Account Balance" (float). Stability is required for secondary sort by "Date". Ample RAM available.
3. An embedded system in a Buea weather station sorts 100 temperature logs (structs: float temp, int timestamp) stored in Flash memory. Write cycles to Flash must be minimised.
4. A student's GCE project sorts a linked list of 500 strings (names) in Python. The language's built-in `sorted()` uses Timsort. Explain why Timsort fits.

**SOLUTION**

Detailed Solution

1. Algorithm: Insertion Sort.
   - Small $n = 20$: Insertion Sort has minimal overhead, faster than $O(n \log n)$ algorithms for tiny $n$.
   - Nearly Sorted: Best case $O(n)$ adaptive behaviour.
   - RAM 2KB / In-place: $O(1)$ extra space. No recursion stack overflow risk.
2. Algorithm: Merge Sort (or Timsort).
   - Large $n = 10^6$: Requires $O(n \log n)$ guaranteed. Quick Sort worst case $O(n^2)$ risky for server.
   - Stability Required: Merge Sort is stable. Quick Sort / Heap Sort are not.
   - Ample RAM: $O(n)$ auxiliary array acceptable.
3. Algorithm: Selection Sort.
   - Minimise Writes (Flash): Selection Sort performs at most $n - 1 = 99$ swaps (writes). Insertion Sort shifts ($O(n^2)$ writes). Bubble Sort swaps ($O(n^2)$ writes).
   - Small $n = 100$: $O(n^2)$ time acceptable.

- In-place: $O(1)$ space.

4. Timsort Justification:
   - Hybrid (Merge + Insertion): Uses Insertion Sort for small runs ($n < 64$), Merge Sort for merging runs. Optimal for all sizes.
   - Adaptive (Natural Runs): Detects existing ordered subsequences (ascending/descending) in the linked list. Exploits partial order → often $O(n)$.
   - Stable: Preserves order of equal names (important for secondary keys).
   - Linked List Friendly: Merge Sort works naturally on linked lists ($O(1)$ merge space). Timsort leverages this.

**METHOD** **Complexity Analysis: The Master Theorem (for Divide & Conquer Sorts)**

Use: Analyse recursive algorithms like Merge Sort and Quick Sort. Recurrence: $T(n) = aT(n/b) + f(n)$.

- $a$: Number of subproblems.
- $n/b$: Size of each subproblem.
- $f(n)$: Cost to divide + combine.

Three Cases (simplified):

1. If $f(n) = O(n^{\log_b a - \epsilon})$: $T(n) = \Theta(n^{\log_b a})$ (Leaves dominate).
2. If $f(n) = \Theta(n^{\log_b a} \log^k n)$: $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$ (Balanced).
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ and $af(n/b) \leq cf(n)$: $T(n) = \Theta(f(n))$ (Root dominates).

Application to Syllabus Sorts:

- Merge Sort: $T(n) = 2T(n/2) + \Theta(n)$. $a = 2, b = 2, \log_2 2 = 1$. $f(n) = \Theta(n^1)$. Case 2 ($k = 0$). → $\Theta(n \log n)$.
- Quick Sort (Average): $T(n) = 2T(n/2) + \Theta(n) \rightarrow \Theta(n \log n)$.
- Quick Sort (Worst): $T(n) = T(n-1) + \Theta(n) \rightarrow \Theta(n^2)$ (Master Theorem not directly applicable, use substitution/tree).

**WORKED EXAMPLE 11** **Merge Sort Recurrence and Trace**

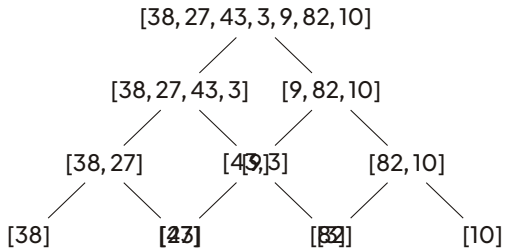
1. Write the recurrence relation for the worst-case time complexity of Merge Sort. Solve it using the Master Theorem, stating $a$, $b$, $f(n)$ and the case used.

2. Trace the divide phase (splitting) and merge phase for the array $A = [38, 27, 43, 3, 9, 82, 10]$. Draw the recursion tree showing subarrays at each level.
3. How many extra array cells (auxiliary space) are needed in total during the merge of the final two halves for this array of size 7?

**SOLUTION**

Detailed Solution

1. Recurrence: $T(n) = 2T(n/2) + \Theta(n)$ for $n > 1$, $T(1) = \Theta(1)$.
   - $a = 2$ (two recursive calls).
   - $b = 2$ (each half size).
   - $f(n) = \Theta(n)$ (merge step linear).
   - $\log_b a = \log_2 2 = 1$.
   - $f(n) = \Theta(n) = \Theta(n^{\log_b a})$. Case 2 with $k = 0$.
   - Solution: $T(n) = \Theta(n \log n)$.
2. Recursion Tree Trace:




   Merge Phase (Bottom-Up):
   - Merge [38], [27] → [27, 38]
   - Merge [43], [3] → [3, 43]
   - Merge [27, 38], [3, 43] → [3, 27, 38, 43]
   - Merge [82], [10] → [10, 82]
   - Merge [9], [10, 82] → [9, 10, 82]
   - Merge [3, 27, 38, 43], [9, 10, 82] → [3, 9, 10, 27, 38, 43, 82]
3. Auxiliary Space: Standard Merge Sort merges two halves of total size $n$ into a temporary array of size $n$. For $n = 7$, final merge needs 7 extra cells. (Note: Total extra space allocated over lifetime can be $O(n \log n)$ if not reused, but standard implementation allocates one temp array of size $n \rightarrow O(n)$ space complexity).

METHOD **Quick Sort: Partitioning (Lomuto Scheme) and Pivot Choice**

Core: Partition array around a pivot. Elements < pivot left, > pivot right. Pivot ends in final sorted position. Recurse on left/right parts. Lomuto Partition (Pivot = Last Element $A[high]$):

1. $pivot \leftarrow A[high]$.
2. $i \leftarrow low - 1$ (index of smaller element).
3. For $j \leftarrow low$ to $high - 1$:
4. If $A[j] \leq pivot$: $i \leftarrow i + 1$; Swap $A[i], A[j]$.
5. Swap $A[i + 1], A[high]$ (Place pivot).
6. Return $i + 1$ (pivot index).

Pivot Choice Strategies (Critical for Performance):

- Fixed (First/Last): Simple. Worst case $O(n^2)$ on sorted/reverse sorted arrays.
- Random: Expected $O(n \log n)$. Defeats adversarial input.
- Median-of-Three (First, Mid, Last): Robust practical choice. Avoids worst case on sorted data.

Properties: In-place ($O(\log n)$ stack), Unstable, Cache-friendly.

WORKED EXAMPLE 12 **Quick Sort Partition Trace and Worst Case**

Array $A = [10, 80, 30, 90, 40, 50, 70]$ (indices 0..6). Use Lomuto Partition with pivot = last element (70).

1. Trace the partition loop. Show a table: $j$, $A[j]$, Condition ($A[j] \leq 70$?), $i$ after inc, Array state after potential swap.
2. State the final array after partition and the returned pivot index.
3. Show the array configuration (values) that would cause worst-case $O(n^2)$ behaviour for Quick Sort with "Last Element" pivot on an array of size 5. Explain why.

SOLUTION

Detailed Solution

1. Partition Trace ($pivot = 70$):

| $j$ | $A[j]$ | $A[j] \le 70$? | $i$ (before) | Action | Array State |
|---|---|---|---|---|---|
| Init | - | - | -1 | - | [10, 80, 30, 90, 40, 50, 70] |
| 0 | 10 | Yes | $-1 \to 0$ | Swap $A[0], A[0]$ | [10, 80, 30, 90, 40, 50, 70] |
| 1 | 80 | No | 0 | None | [10, 80, 30, 90, 40, 50, 70] |
| 2 | 30 | Yes | $0 \to 1$ | Swap $A[1], A[2]$ | [10, **30**, **80**, 90, 40, 50, 70] |
| 3 | 90 | No | 1 | None | [10, 30, 80, 90, 40, 50, 70] |
| 4 | 40 | Yes | $1 \to 2$ | Swap $A[2], A[4]$ | [10, 30, **40**, 90, **80**, 50, 70] |
| 5 | 50 | Yes | $2 \to 3$ | Swap $A[3], A[5]$ | [10, 30, 40, **50**, 80, **90**, 70] |
| End | - | - | 3 | Swap $A[4], A[6]$ (Pivot) | [10, 30, 40, 50, **70**, 90, 80] |

2. Result: Array = [10, 30, 40, 50, 70, 90, 80]. Pivot Index = 4. Left: [10, 30, 40, 50] (< 70). Right: [90, 80] (> 70).

3. Worst Case Config (Size 5, Pivot=Last): Already Sorted Ascending: [10, 20, 30, 40, 50].

   - Call 1: Pivot=50. Partition: Left size 4, Right size 0. Comparisons: 4.
   - Call 2: Pivot=40. Left size 3, Right 0. Comparisons: 3.
   - Call 3: Pivot=30. Left size 2. Comparisons: 2.
   - Call 4: Pivot=20. Left size 1. Comparisons: 1.
   - Total Comparisons: $4 + 3 + 2 + 1 = 10 = n(n-1)/2 = \Theta(n^2)$.

   Why? Pivot is always the maximum element. Partition is maximally unbalanced ($n-1$ and 0). Recursion depth $n$.

# 8 EXERCISES

## 8.1 I check my knowledge

**EXERCISE 5** **MCQ on Searching**

Which of the following statements about linear search and binary search is correct?

1. Linear search requires the array to be sorted.
2. Binary search has a worst-case time complexity of $O(n)$.
3. Linear search has a worst-case time complexity of $O(n)$.
4. Binary search can be applied directly on an unsorted array.

**EXERCISE 6** **True/False with Justification**

State whether each statement is true or false. Justify your answer.

1. Bubble sort always performs the same number of comparisons regardless of the initial order of the elements.
2. Selection sort is a stable sorting algorithm.
3. Insertion sort performs well on nearly sorted lists.
4. The best-case time complexity of binary search is $O(1)$.

**EXERCISE 7** **Definitions**

Define the following terms:

1. Searching algorithm
2. Sorting algorithm
3. Stable sort
4. In-place sort

**EXERCISE 8** **Direct Calculations**

An array contains 500 000 elements.

1. What is the maximum number of comparisons required by linear search in the worst case?
2. What is the maximum number of comparisons required by binary search in the worst case? (Give the exact integer value.)

## 8.2 I practise

**EXERCISE 9** **Bubble Sort Trace**

Given the array `[42, 17, 8, 29, 13]`, show the state of the array after each pass of the standard bubble sort algorithm (with the optimization that stops if no swaps occur in a pass). Also state the total number of comparisons and swaps performed.

**EXERCISE 10** **Binary Search Trace**

Consider the sorted array `[3, 7, 11, 18, 25, 31, 40, 46, 52, 60]`. Perform a binary search for the key `31` and then for the key `20`. For each search, show the values of `low`, `high`, and `mid` at each step, and indicate whether the key is found.

**EXERCISE 11** **Code Completion – Linear Search**

Complete the following Python function that performs linear search. It should return the index of the first occurrence of `key` in `arr`, or `-1` if not found.

```
def linear_search(arr, key):
    # TODO: complete the function
    pass
```

**EXERCISE 12** **Error Hunting – Binary Search**

The following pseudocode for binary search contains three bugs. Identify and correct them.

```
function binary_search(arr, key):
    low = 0
```

```
    high = length(arr)      // Bug 1
    while low < high:       // Bug 2
        mid = (low + high) / 2
        if arr[mid] == key:
            return mid
        else if arr[mid] < key:
            low = mid       // Bug 3
        else:
            high = mid
    return -1
```

## 8.3 I prepare for the GCE

**EXERCISE 13** **Comparison Problem – Cybercafé Database**

A cybercafé in Douala maintains a database of 500,000 customer records. The manager wants to search for a customer by their ID number.

1. Calculate the maximum number of comparisons required if linear search is used. [2 marks]
2. Calculate the maximum number of comparisons required if binary search is used (assume the records are sorted by ID). [2 marks]
3. If each comparison takes 0.1 microseconds, estimate the worst-case search time for each method. [2 marks]
4. Based on your calculations, which method is feasible for real-time use? Justify your answer. [2 marks]

**EXERCISE 14** **Algorithm Choice Essay**

For each of the following scenarios, choose the most appropriate searching or sorting strategy and justify your choice.

1. A small unsorted list of 20 student names that needs to be sorted once. [3 marks]
2. A huge sorted register of 1,000,000 telephone numbers that is searched frequently. [3 marks]
3. A nearly sorted list of 500 examination scores that arrives daily and must be sorted. [3 marks]
4. A list of 100 items that must be searched only once and is not sorted. [3 marks]

**EXERCISE 15** **Structured GCE-Style Question**

1. Define the term sorting. [2 marks]
2. Trace the selection sort algorithm on the array `[64, 25, 12, 22, 11, 90]`. Show the array after each pass. [6 marks]
3. State the time complexity of selection sort in the best, average, and worst cases. [3 marks]
4. Name one sorting algorithm that is asymptotically faster than selection sort for large datasets. [1 mark]

# 9 SOLUTIONS TO THE EXERCISES

## 9.1 I check my knowledge

**SOLUTION** **Exercise 1 — MCQ on Searching**

Correct answer: (c) Linear search has a worst-case time complexity of $O(n)$.
Justification of each option:

1. False. Linear search examines elements sequentially from the first to the last; it works on any array, sorted or not. This absence of a precondition is its main advantage for small or unsorted datasets.
2. False. Binary search halves the search interval at each step, so its worst-case complexity is $O(\log_2 n)$, not $O(n)$.
3. True. In the worst case (key absent or located at the last position), linear search compares the key with all $n$ elements, performing exactly $n$ comparisons, hence $O(n)$.
4. False. Binary search is only correct if the array is **sorted** (ascending or descending). On an unsorted array, eliminating half of the elements based on a comparison with the middle element may discard the key, leading to incorrect results.

**SOLUTION** **Exercise 2 — True/False with Justification**

1. False (for the optimized version). Standard bubble sort with an "early exit" flag stops as soon as a complete pass performs no swap. On an already sorted array it performs only $n-1$ comparisons (one pass), whereas on a reverse-sorted array it performs $\frac{n(n-1)}{2}$ comparisons. Only the unoptimized version always performs the same number of comparisons regardless of input order.
2. False. Selection sort is not stable: swapping the minimum element found in the unsorted part with the element at the current position can move an element over another element with an equal key, changing their relative order. Example: sorting `[2a, 2b, 1]` (where `2a` and `2b` are distinct records with equal key 2) gives `[1, 2b, 2a]`, reversing the order of the two 2s.
3. True. Insertion sort is **adaptive**: on a nearly sorted list, each element is already close to its final position, so very few shifts are needed. Its best-case complexity is $O(n)$ and it approaches this performance on nearly sorted data.
4. True. In the best case, the key is exactly at the middle position on the very first probe: a single comparison suffices, hence $O(1)$.

**SOLUTION** **Exercise 3 — Definitions**

1. Searching algorithm: a systematic procedure that determines whether a given value (the **key**) is present in a collection of data and, if so, returns its position (or the associated record).
2. Sorting algorithm: a systematic procedure that rearranges the elements of a collection into a specified order (ascending or descending) according to a comparison key.
3. Stable sort: a sorting algorithm that preserves the relative order of elements having equal keys: if two equal elements $A$ and $B$ appear with $A$ before $B$ in the input, then $A$ still appears before $B$ in the output.
4. In-place sort: a sorting algorithm that requires only a constant amount $O(1)$ of extra memory, rearranging the elements within the original array itself (apart from a few temporary variables for swapping).

**SOLUTION** **Exercise 4 — Direct Calculations**

1. Linear search (worst case): the key is compared with every element, so the maximum number of comparisons is exactly

$$n = 500\,000 \text{ comparisons.}$$

2. Binary search (worst case): the maximum number of comparisons for a successful or unsuccessful search is $\lfloor \log_2 n \rfloor + 1$. Compute:

$$2^{18} = 262\,144 \quad \text{and} \quad 2^{19} = 524\,288.$$

Since $2^{18} < 500\,000 \le 2^{19}$, we have $\lfloor \log_2 500\,000 \rfloor = 18$, hence the worst case is

$$18 + 1 = 19 \text{ comparisons.}$$

Remark: binary search reduces half a million checks to at most 19 — this is why sorted data and binary search are so valuable in practice.

## 9.2 I practise

**SOLUTION** **Exercise 5 — Bubble Sort Trace**

Array: `[42, 17, 8, 29, 13]`, $n = 5$. We use the optimized version with a `swapped` flag.

Pass 1 (compare adjacent pairs from index 0 to 3):

- 42 > 17: swap → `[17, 42, 8, 29, 13]`
- 42 > 8: swap → `[17, 8, 42, 29, 13]`
- 42 > 29: swap → `[17, 8, 29, 42, 13]`
- 42 > 13: swap → `[17, 8, 29, 13, 42]`

After pass 1: `[17, 8, 29, 13, 42]` — 4 comparisons, 4 swaps. Largest element (42) bubbled to the end.
Pass 2 (last element 42 is in place; compare indices 0 to 2):

- 17 > 8: swap → `[8, 17, 29, 13, 42]`
- 17 < 29: no swap
- 29 > 13: swap → `[8, 17, 13, 29, 42]`

After pass 2: `[8, 17, 13, 29, 42]` — 3 comparisons, 2 swaps.
Pass 3 (last two elements sorted; compare indices 0 to 1):

- 8 < 17: no swap
- 17 > 13: swap → `[8, 13, 17, 29, 42]`

After pass 3: `[8, 13, 17, 29, 42]` — 2 comparisons, 1 swap.
Pass 4 (last three elements sorted; compare index 0 only):

- 8 < 13: no swap. No swap occurred during the whole pass ⇒ algorithm stops early.

After pass 4: `[8, 13, 17, 29, 42]` — 1 comparison, 0 swaps.
Totals: 4 + 3 + 2 + 1 = 10 comparisons and 4 + 2 + 1 = 7 swaps. The array is sorted: `[8, 13, 17, 29, 42]`.

SOLUTION **Exercise 6 — Binary Search Trace**

Array (indices 0 to 9): `[3, 7, 11, 18, 25, 31, 40, 46, 52, 60]`.
Search for key 31:

| Step | `low` | `high` | `mid` | `arr[mid]` | Action |
|---|---|---|---|---|---|
| 1 | 0 | 9 | 4 | 25 | 25 < 31: `low` = 5 |
| 2 | 5 | 9 | 7 | 46 | 46 > 31: `high` = 6 |
| 3 | 5 | 6 | 5 | 31 | Found! Return index 5 |

The key `31` is found at index 5 after 3 comparisons.
Search for key 20:

| Step | low | high | mid | arr[mid] | Action |
|---|---|---|---|---|---|
| 1 | 0 | 9 | 4 | 25 | 25 > 20: high = 3 |
| 2 | 0 | 3 | 1 | 7 | 7 < 20: low = 2 |
| 3 | 2 | 3 | 2 | 11 | 11 < 20: low = 3 |
| 4 | 3 | 3 | 3 | 18 | 18 < 20: low = 4 |

Now low = 4 > high = 3: the loop ends. The key 20 is not found; the algorithm returns -1 after 4 comparisons.

**SOLUTION** **Exercise 7 — Code Completion (Linear Search)**

A correct Python implementation:

```
def linear_search(arr, key):
  for i in range(len(arr)):
    if arr[i] == key:
      return i
  return -1
```

Explanation: the loop scans the indices from 0 to len(arr)-1. As soon as an element equals key, its index is returned immediately, which guarantees that the first occurrence is returned. If the loop finishes without a match, the function returns -1. Worst-case complexity: $O(n)$; best case: $O(1)$ (key at index 0).

**SOLUTION** **Exercise 8 — Error Hunting (Binary Search)**

Bug 1 — wrong upper bound: high = length(arr) is out of range, since valid indices run from 0 to length(arr) - 1. This can cause an index-out-of-range access when mid equals length(arr). Correction: high = length(arr) - 1.
Bug 2 — wrong loop condition: while low < high skips the case low = high, where a single unexamined element remains (e.g. a one-element array would never be checked). Correction: while low <= high.
Bug 3 — no progress on the lower bound: low = mid does not exclude arr[mid], which has already been tested and found not to match. Combined with integer division rounding down, the interval may never shrink, causing an infinite loop (e.g. low = 3, high = 4 gives mid = 3 forever). Correction: low = mid + 1. (Symmetrically, high = mid - 1 in the other branch.)
Corrected pseudocode:

```
function binary_search(arr, key):
  low = 0
  high = length(arr) - 1
  while low <= high:
    mid = (low + high) div 2
    if arr[mid] == key: return mid
    else if arr[mid] < key: low = mid + 1
    else: high = mid - 1
  return -1
```

## 9.3 I prepare for the GCE

**SOLUTION** **Exercise 9 — Comparison Problem (Cybercafé Database)**

(a) Linear search [2 marks]: worst case = key absent or last record:

$$500\,000 \text{ comparisons.}$$

[1 mark for the principle $n$ comparisons; 1 mark for the numerical answer.]
(b) Binary search [2 marks]: worst case = $\lfloor \log_2 500\,000 \rfloor + 1$. Since $2^{18} = 262\,144 < 500\,000 \le 2^{19} = 524\,288$:

$$\lfloor \log_2 500\,000 \rfloor + 1 = 18 + 1 = \mathbf{19} \text{ comparisons.}$$

[1 mark for the logarithmic formula; 1 mark for the exact value 19.]
(c) Search times [2 marks]: each comparison takes $0.1\,\mu s = 10^{-7}$ s.

- Linear search: $500\,000 \times 0.1\,\mu s = 50\,000\,\mu s = \mathbf{50}$ ms (0.05 s).
- Binary search: $19 \times 0.1\,\mu s = \mathbf{1.9}\,\mu s$.

[1 mark per correct time with unit.]
(d) Conclusion [2 marks]: Binary search is the feasible method for real-time use: at under 2 microseconds per query it is effectively instantaneous, even with many simultaneous users. Linear search at 50 ms per query would accumulate noticeable delays under heavy load (e.g. 100 queries ≈ 5 s). The one-time cost of keeping records sorted by ID is amply repaid. [1 mark for the correct choice; 1 mark for a justification referring to the computed times.]
Examiner's expectations: exact integer 19 (not just "$\log_2 n$"), correct unit conversions, and a conclusion that explicitly compares the two computed times.

**SOLUTION** **Exercise 10 — Algorithm Choice Essay**

(a) 20 student names, sorted once [3 marks]: Insertion sort (or selection sort). The list is tiny, so asymptotic efficiency is irrelevant; a simple $O(n^2)$ algorithm is easy to implement correctly and runs instantly. Insertion sort is preferred as it is stable and simple. [1 mark choice, 2 marks justification mentioning small $n$ and simplicity.]
(b) 1,000,000 sorted telephone numbers, searched frequently [3 marks]: Binary search. The data is already sorted, so the precondition is satisfied; each search costs at most $\lfloor \log_2 10^6 \rfloor + 1 = 20$ comparisons instead of up to $10^6$ for linear search. Since searches are frequent, the saving is enormous. [1 mark choice, 2 marks justification with the logarithmic argument and the "frequent searches" observation.]
(c) 500 nearly sorted examination scores arriving daily [3 marks]: Insertion sort. It is **adaptive**: on nearly sorted data it performs close to the best case $O(n)$, because each element is shifted only a few positions. Bubble sort with early exit would also be acceptable, but insertion sort is the classic answer. [1 mark choice, 2 marks justification using adaptivity / nearly sorted best case.]
(d) 100 unsorted items, searched only once [3 marks]: Linear search. Sorting first would cost $O(n \log n)$ at best (or $O(n^2)$ with a simple sort), which is far more than the $n = 100$ comparisons of a single linear scan. Sorting only pays off when many searches will reuse the sorted structure. [1 mark choice, 2 marks justification comparing one-off search cost with sort-then-search cost.]

**SOLUTION** **Exercise 11 — Structured GCE-Style Question**

(a) Definition of sorting [2 marks]: sorting is the process of rearranging the elements of a collection into a specified order (ascending or descending) according to a comparison key. [1 mark "rearranging elements"; 1 mark "into a defined order".]
(b) Trace of selection sort on `[64, 25, 12, 22, 11, 90]` [6 marks]:
Principle: at pass $i$ (starting from 0), find the minimum of the unsorted part (indices $i$ to $n - 1$) and swap it into position $i$.

- Pass 1 ($i = 0$): minimum of `[64, 25, 12, 22, 11, 90]` is 11 (index 4); swap with 64 → `[11, 25, 12, 22, 64, 90]`
- Pass 2 ($i = 1$): minimum of `[25, 12, 22, 64, 90]` is 12 (index 2); swap with 25 → `[11, 12, 25, 22, 64, 90]`
- Pass 3 ($i = 2$): minimum of `[25, 22, 64, 90]` is 22 (index 3); swap with 25 → `[11, 12, 22, 25, 64, 90]`
- Pass 4 ($i = 3$): minimum of `[25, 64, 90]` is 25, already in place → `[11, 12, 22, 25, 64, 90]`
- Pass 5 ($i = 4$): minimum of `[64, 90]` is 64, already in place → `[11, 12, 22, 25, 64, 90]`

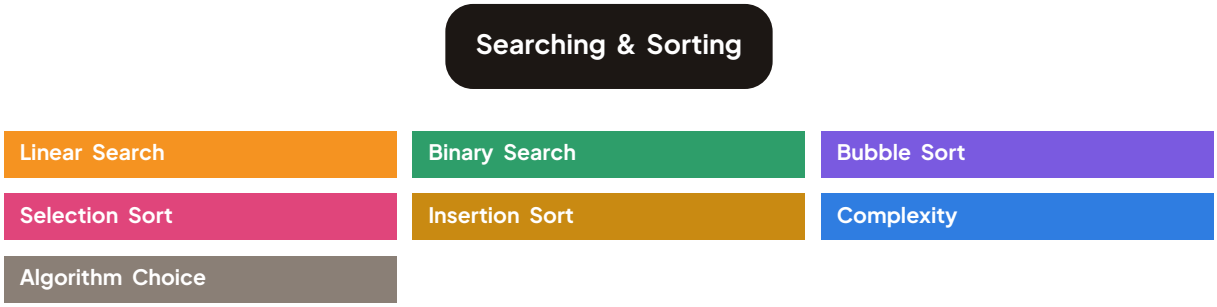
Final sorted array: `[11, 12, 22, 25, 64, 90]`. [1 mark per correct pass, up to 5 marks; 1

mark for the correct final array. The examiner expects the minimum to be identified explicitly at each pass.]
(c) Time complexities of selection sort [3 marks]:

- Best case: $O(n^2)$
- Average case: $O(n^2)$
- Worst case: $O(n^2)$

Selection sort always performs $\frac{n(n-1)}{2}$ comparisons whatever the initial order, since finding the minimum of the unsorted part requires scanning it entirely. The number of swaps is at most $n-1$. [1 mark per case.]
(d) A faster algorithm [1 mark]: Merge sort (or quick sort), with complexity $O(n \log n)$ on average (guaranteed for merge sort). [1 mark for any valid $O(n \log n)$ algorithm.]

# 10 SUMMARY SHEET




Chapter CSC03 mind map: Searching and Sorting algorithms

FICHE BILAN **L'essentiel à connaître pour l'examen**

Key Definitions

- **Linear Search**: sequential scan of an array until the target is found or the end is reached.
- **Binary Search**: divide-and-conquer search on a sorted array, halving the search interval each step.
- **Bubble Sort**: repeatedly swaps adjacent out-of-order elements, bubbling the largest to the end of the unsorted portion.
- **Selection Sort**: repeatedly selects the minimum (or maximum) from the unsorted part and swaps it into its final position.
- **Insertion Sort**: builds the sorted array one element at a time by inserting each new element into its correct place among the already sorted elements.
- **Time Complexity**: worst-case number of basic operations as a function of input size $n$.
- **Space Complexity**: extra memory used besides the input array.
- **Stability**: a sorting algorithm is stable if equal elements retain their original relative order.
- **Adaptive**: an algorithm that performs better on partially sorted data (e.g., Insertion Sort, Bubble Sort with early exit).

Complexity Laws (must know for GCE)

- Linear Search: $O(n)$ time, $O(1)$ space.

- Binary Search (iterative): $O(\log n)$ time, $O(1)$ space; recursive version uses $O(\log n)$ stack space.
- Bubble Sort: $O(n^2)$ worst/average time, $O(1)$ space; best case $O(n)$ with early-exit flag.
- Selection Sort: $\Theta(n^2)$ comparisons always, $O(1)$ space; not stable.
- Insertion Sort: $O(n^2)$ worst/average, $O(n)$ best (already sorted), $O(1)$ space; stable and adaptive.

Methods / Pseudocode Patterns

- Linear Search: `for i = 0 to n-1: if A[i] == key return i; return -1;`
- Binary Search (iterative): `low=0; high=n-1; while low <= high: mid=(low+high)/2; if A[mid]==key return mid; else if A[mid]<key low=mid+1; else high=mid-1; return -1;`
- Bubble Sort: `for i=0 to n-2: swapped=false; for j=0 to n-2-i: if A[j]>A[j+1] swap; swapped=true; if not swapped break;`
- Selection Sort: `for i=0 to n-2: minIdx=i; for j=i+1 to n-1: if A[j]<A[minIdx] minIdx=j; swap A[i],A[minIdx];`
- Insertion Sort: `for i=1 to n-1: key=A[i]; j=i-1; while j>=0 and A[j]>key: A[j+1]=A[j]; j--; A[j+1]=key;`

Common Mistakes (GCE traps)

- Forgetting that Binary Search requires a sorted array; applying it on unsorted data yields incorrect results.
- Off-by-one errors in loop bounds (e.g., using <= vs < in Binary Search condition).
- Confusing the number of passes in Bubble Sort: exactly $n-1$ passes, inner loop shrinks by one each pass.
- Thinking Selection Sort is stable; it is not stable because long-distance swaps can re-order equal elements.
- Assuming Insertion Sort is always $O(n^2)$; best case is $O(n)$ when data is already or nearly sorted.
- Not stating space complexity for recursive Binary Search ($O(\log n)$ stack frames).
- Omitting the early-exit optimisation in Bubble Sort when asked for best-case analysis.

GCE Examination Tips

- Always write the **worst-case**, **best-case**, and **average-case** complexities when asked.
- For trace questions, show a clear **trace table** with each pass/iteration, indices, and array state.

- When comparing algorithms, mention both time and space complexity, and stability if relevant.
- Pseudocode must be language-independent: use `for`, `while`, `if`, `swap`, `return`.
- In essay-type questions, justify the choice of algorithm for a given scenario (e.g., small $n$ or nearly sorted → Insertion Sort; large sorted data → Binary Search).
- Remember that Bubble Sort with early exit can detect an already sorted array in $O(n)$ time.
- If asked to implement a search on unsorted data, Linear Search is the only correct choice unless you sort first (which adds $O(n \log n)$ or $O(n^2)$ overhead).

**Signed • OnBuch+**
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+